

AI-Driven Sample Space Analysis for Offshore Riser Behavior Modeling

Technical Presentation

Machine learning pipelines for predicting fatigue damage, significant wave height, and wave heading from environmental loading and vessel motion sensor data.

3
Pipelines

40,000+
Total Experiments

8
Production Models

156
Features Evaluated

Agenda

Section 1

Fatigue Damage Prediction

Slides 4–17

Predicting fatigue damage rates at three riser locations (Top, OD, ID) from 5 environmental parameters. Covers data characterization, feature engineering with 5 wave energy proxy candidates, 198 ML experiments, 3,327 DL architecture searches, and the final 46-neuron MLP.

Section 2

Significant Wave Height Prediction

Slides 18–27

Estimating significant wave height (H_s) from vessel motion sensor data using 6 standard deviation features. Covers 245 model selection experiments across 8 algorithms, Random Forest model mechanics, and the production pipeline achieving $R^2 = 0.9992$.

Section 3a

Wave Heading — Circular Regression

Slides 28–45

Predicting wave direction (0° – 360°) using sin/cos target encoding and 22 features selected from 156 candidates through 166 rounds of Recursive Feature Elimination ($\sim 1.08\text{M}$ model fits). Covers circular error metrics, the RFE methodology, and ExtraTreesRegressor performance.

Section 3b

Wave Heading — Hierarchical Quadrant Cascade

Slides 46–69

Two-stage approach: classify into 4 quadrants (100% test accuracy), then predict heading within each quadrant using specialized regressors. Covers 171 classification RFE rounds, 36,752 regression experiments, and per-quadrant performance analysis.

Each section follows the same logical sequence: data → features → model selection → performance evaluation → production configuration.

Introduction: Project Scope

This presentation covers three machine learning pipelines developed for an offshore riser system operating in the Dorado field. Each pipeline addresses a different prediction task, and together they provide the environmental characterization needed for riser fatigue and response analysis.

Fatigue Damage Prediction

5 environmental inputs
→ 6 physics features
→ 3 damage outputs
(Top, OD, ID)

88,560 training sea states
3,525 experiments
46-neuron MLP
 $R^2 = 0.8086$

Significant Wave Height Prediction

6 motion signals
→ 6 STD features
→ 1 Hs output (ft)

3,888 simulation CSVs
245 experiments
Random Forest (100 trees)
 $R^2 = 0.9992$

Wave Heading Prediction

7 motion signals
→ 156 candidate features
→ 22 or 36+21 selected
→ heading (°)

3,888 simulation CSVs
~1.08M + 36,752 experiments
2 approaches compared
Max error = 12.50°

For each pipeline, we will present:

- Data characterization and input/output specification with units
- Feature engineering rationale and physics interpretation
- Model selection process with exact experiment counts and metrics
- Production model architecture with visualization
- Production configuration: commands, artifacts, and source code references

Section 1: Fatigue Damage Prediction

5 environmental inputs → 6 physics-based features → 3 damage outputs (Top, OD, ID)

88,560 training sea states | 3,525 total experiments (198 ML + 3,327 DL)

Production model: 46-neuron MLP | Combined $R^2 = 0.8086$

Data Overview

Training Dataset: 88,560 sea states from hydrodynamic simulation

Test Dataset: 223 operational sea states (Dorado field conditions)

- Wave direction: 0° to 350° at 70° intervals (6 directions)
- Vessel offset: 5 ft intervals
- Damage values span 6 orders of magnitude: 1×10^{-9} to 1×10^{-3} damage

5 Raw Inputs (Field Units)

Input	Unit	Description
Direction	deg	Wave approach direction (compass bearing)
Hs	ft	Significant wave height
Tp	s	Peak spectral wave period
East	ft	Vessel offset east of riser centerline
North	ft	Vessel offset north of riser centerline

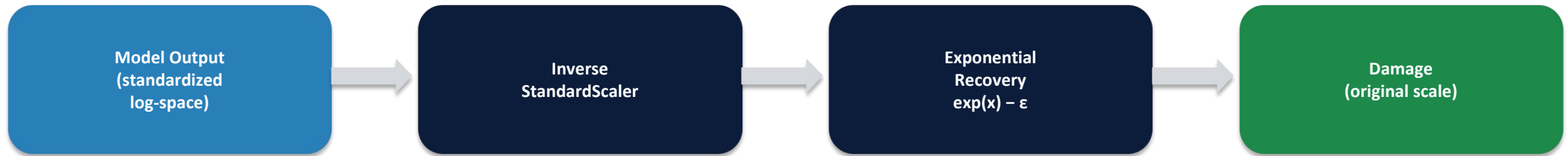
3 Prediction Targets

Output	Unit	Description
Predicted_Top	damage	Fatigue at riser top (hang-off) connection
Predicted_OD	damage	Fatigue at outer diameter critical zone
Predicted_ID	damage	Fatigue at inner diameter critical zone

The Log-Transform Challenge

- Damage values span 6 orders of magnitude (1×10^{-9} to 1×10^{-3} damage)
- Direct regression fails — the model learns only the dominant magnitude scale and ignores smaller values
- Solution: logarithmic compression $\ln(\text{damage} + 1 \times 10^{-6})$
- This compresses the range from $[1 \times 10^{-9}, 1 \times 10^{-3}]$ to approximately $[-20, -7]$
- The model can now learn uniformly across all damage magnitudes
- Epsilon = 1×10^{-6} prevents $\ln(0)$ and sets the noise floor

Inverse Recovery at Prediction Time:



Feature Engineering: 5 Inputs → 6 Physics Features

#	Feature	Source Transform	Unit / Range	Physical Meaning
1	direction_sin	$\sin(\text{Direction}^\circ)$	$[-1, 1]$	East-west component of wave direction
2	direction_cos	$\cos(\text{Direction}^\circ)$	$[-1, 1]$	North-south component of wave direction
3	angle_sin	$\sin(\text{atan2}(\text{East}, \text{North}))$	$[-1, 1]$	East-west component of vessel offset angle
4	angle_cos	$\cos(\text{atan2}(\text{East}, \text{North}))$	$[-1, 1]$	North-south component of vessel offset angle
5	radii	$\sqrt{\text{East}^2 + \text{North}^2}$	ft	Distance from riser centerline
6	hs2_div_tp	H_s^2 / T_p	ft^2/s	Wave energy flux proxy

- Sin/cos encoding: Direction is circular ($0^\circ = 360^\circ$). Direct numerical input creates a false discontinuity — the model would treat 1° and 359° as 358 units apart when they are 2° apart.
- Offset encoding: East/North Cartesian coordinates are converted to polar (radii + angle) to separate distance from direction. The angle is then sin/cos encoded for the same circular reason.
- Wave energy proxy: H_s^2/T_p represents the rate of wave energy delivery per unit time. Selected as the best-performing of 5 candidates tested across 120 experiments.

Wave Energy Proxy Selection: 5 Candidates Tested

120 experiments across Forest Ensemble models to identify the most physically meaningful Hs/Tp combination for fatigue prediction:

Proxy	Formula	Unit	Physical Interpretation
hs_tp	$H_s \times T_p$	ft·s	Wave displacement-duration product — scales with total wave momentum delivered per cycle
hs2_tp	$H_s^2 \times T_p$	ft ² ·s	Energy-duration product — integrates wave energy over one wave period
hs_div_tp	H_s / T_p	ft/s	Wave steepness proxy — rate of vertical displacement per unit time
hs2_div_tp	H_s^2 / T_p	ft ² /s	Wave energy flux — energy delivery rate per unit time (proportional to wave power)
hs_div_tp2	H_s / T_p^2	ft/s ²	Wave-induced acceleration scale — vertical forcing intensity per unit time ²

- hs2_div_tp achieved the best performance: it physically represents wave power (energy is proportional to H_s^2 , and the energy delivery rate is inversely proportional to T_p)
- This is consistent with fatigue mechanics — damage accumulates from repeated loading cycles, and the rate of energy input per unit time directly drives the fatigue loading rate

ML Benchmarking: 198 Experiments

198 total ML experiments:

- Linear Regression: 6 experiments
- Forest Ensemble: 72 experiments
- Forest Ensemble with Log Feature Selection: 120 experiments (testing 5 Hs/Tp proxies)

Best ML Result: RandomForest + PowerTransformer + hs2_div_tp

Target	R ²	Prob Error
Top	0.9442	—
OD	0.4502	—
ID	0.4623	—
Combined	0.6189	48.337

- OD R² = 0.4502 and ID R² = 0.4623 — both below 0.50
- ML tree-based models cannot capture the nonlinear relationship between environmental loading and fatigue damage at the OD and ID locations
- The Top connection (R² = 0.9442) is easier to predict because it experiences more direct wave loading with less geometric attenuation
- Conclusion: ML establishes the optimal feature set but a neural network is needed to model the nonlinear damage accumulation at OD and ID

ML → DL: Narrowing the Search Space

Two-phase search strategy:

Phase 1: ML (198 experiments)

- Test 5 Hs/Tp feature combinations across 120 experiments
- Test 3 algorithm families (Linear, Forest, Forest+Log)
- Establish that hs2_div_tp is the best-performing energy proxy
- Establish that 6-feature input set is optimal
- Identify ML performance ceiling: Combined $R^2 = 0.6189$



Phase 2: DL (3,327 experiments)

- Inherits validated 6-feature input — no re-testing of features
- Focuses only on architecture: neuron count and layer depth
- 91 single-layer widths (10–100 neurons, step 1)
- 324 two-layer combinations (18×18 grid)
- Result: 46-neuron single-layer MLP, Combined $R^2 = 0.8086$

- ML phase validates features. DL phase optimizes architecture. Neither phase repeats the other's work.
- This reduces total search time: feature validation is done once in ML, architecture search is done in DL.
- Total: 3,525 experiments (198 ML + 3,327 DL)

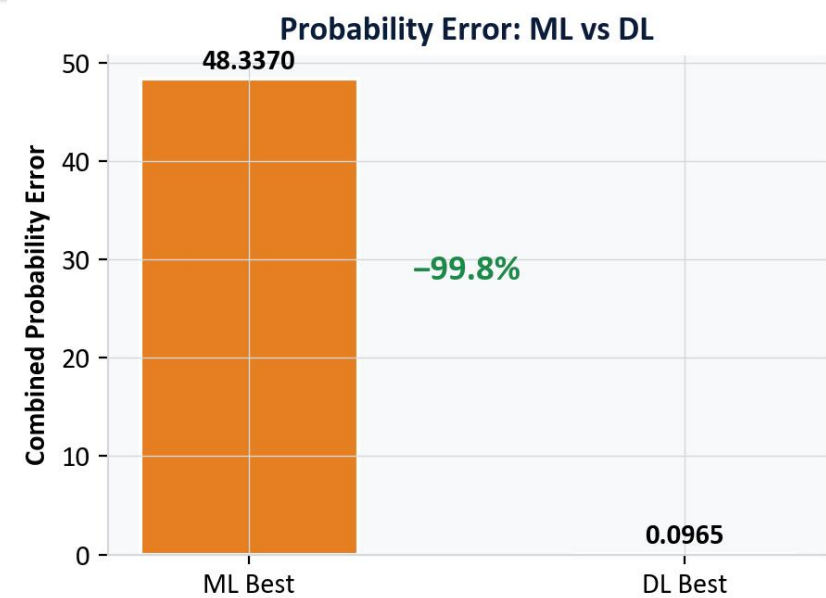
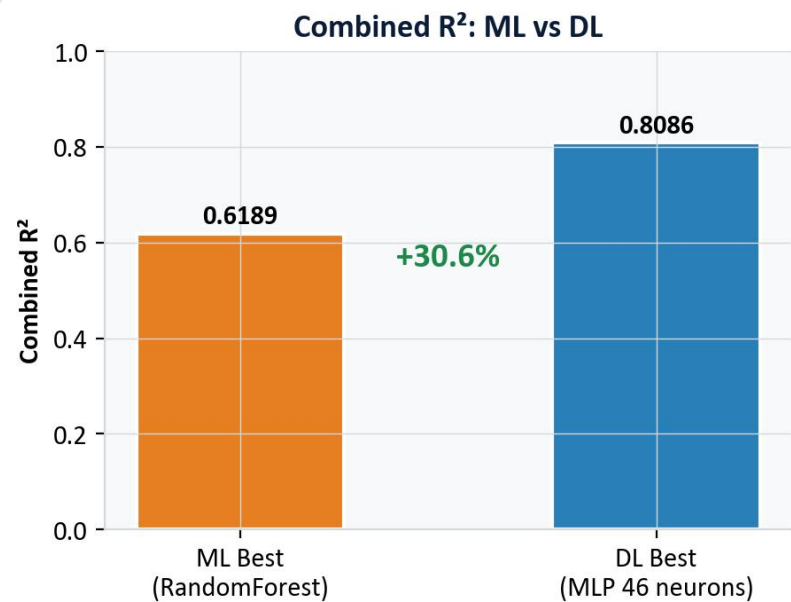
DL Architecture Search: 3,327 Experiments

3,327 MLP architectures tested across 17 experiment tracks:

Track Type	Architecture Range	Count	Description
Single-layer	10 to 100 neurons	91	Every width from 10 to 100 tested (step 1)
Two-layer	18×18 grid	324	All combinations of 2 hidden layer sizes
17 tracks total	Various	3,327	Including learning rate, regularization sweeps

- All architectures use the same validated 6-feature input
- All use: ReLU activation, Adam optimizer, early stopping with 10% validation fraction
- Max iterations: 2,000 (with early stopping)
- Exhaustive single-layer search ensures the global optimum width is found

ML vs DL Comparison: 3,525 Total Experiments



Metric	ML Best	DL Best	Change
Combined R ²	0.6189	0.8086	+30.6%
Prob Error	48.337	0.0965	-99.8%
OD R ²	0.4502	0.7938	+76.3%
ID R ²	0.4623	0.7612	+64.6%
Top R ²	0.9442	0.8709	-7.8%

Top R² decreased slightly because the DL model optimizes Combined performance (all 3 targets jointly), not individual targets. Selected: Single-layer MLP with 46 neurons.

Model Architecture: 46-Neuron MLP

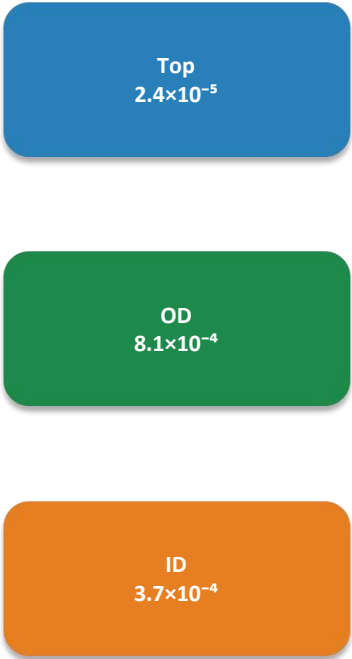
Input Layer (6)



Hidden Layer
(46 neurons, ReLU)



Output Layer (3)
damage



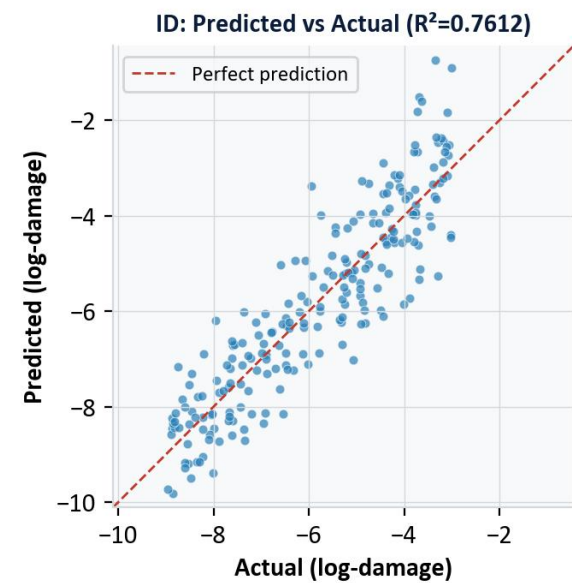
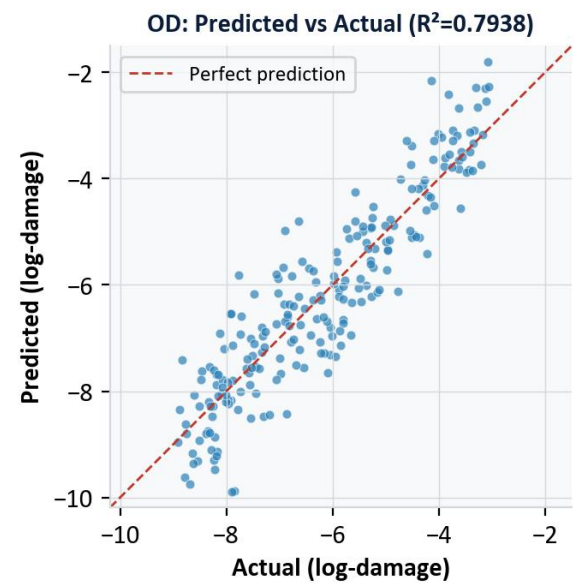
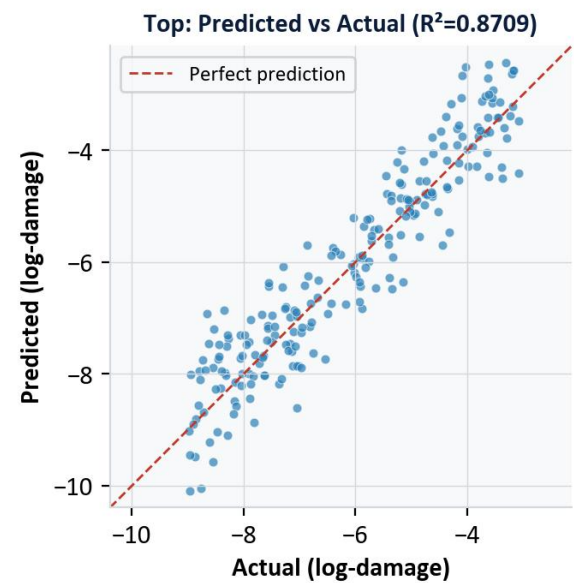
Inverse
Scale + Exp

Hyperparameters: hidden_layer_sizes=[46], activation=relu, solver=adam, max_iter=2000, early_stopping=true, validation_fraction=0.1, random_state=42 | Training: converged in 184 iterations (25.35 s)

Ref: *fatigue_prediction_pipeline/configs/model_config.json*

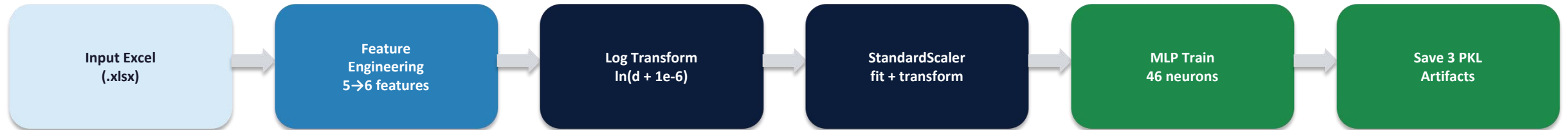
Production Model Performance (223 Dorado Field Sea States)

Target	R ²	Prob Error	Damage Range
Top	0.8709	0.0135	$\sim 1 \times 10^{-9}$ to $\sim 1 \times 10^{-3}$
OD	0.7938	0.0517	$\sim 1 \times 10^{-9}$ to $\sim 1 \times 10^{-3}$
ID	0.7612	0.0313	$\sim 1 \times 10^{-9}$ to $\sim 1 \times 10^{-3}$
Combined	0.8086	0.0965	

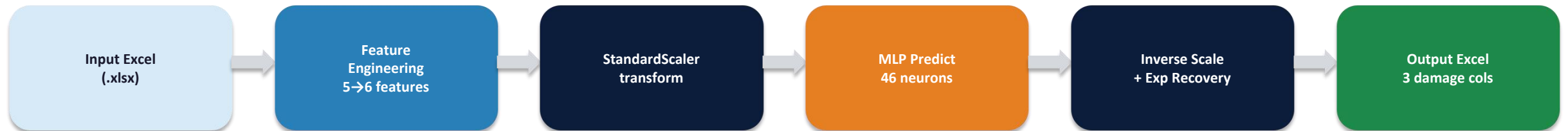


Pipeline Flow: Training and Prediction

Training Pipeline:



Prediction Pipeline:



- Training: Input → 6 features → log-transform targets → fit scalers → train MLP → save artifacts
- Prediction: Input → 6 features → apply saved scaler → MLP predict → inverse-scale → exp-recover → damage output
- All 3 PKL artifacts (model.pkl, input_scaler.pkl, target_scaler.pkl) must come from the same training run

Production Configuration

Commands:

Train: `python main.py --mode train --input "input/Data For Analysis Offset 5Feet Interval and 0 to 350 with 70 Interval.xlsx"`
Predict: `python main.py --mode predict --input "input/new_data.xlsx"`

Production Artifacts:

File	Description
models/model.pkl	Fitted MLPRegressor (46 neurons, ReLU, Adam)
models/input_scaler.pkl	StandardScaler fitted on 6 input features
models/target_scaler.pkl	StandardScaler fitted on 3 log-transformed targets

Model Configuration (configs/model_config.json):

Parameter	Value
hidden_layer_sizes	[46]
activation	relu
solver	adam
max_iter	2000
early_stopping	true
validation_fraction	0.1
random_state	42

Source Code:

- src/features.py — Feature engineering (5 → 6 features)
- src/trainer.py — Training pipeline (FE → Log → Scale → Train)
- src/predictor.py — Inference pipeline (FE → Scale → Predict → Recover)
- configs/feature_config.json — Feature definitions and units
- configs/model_config.json — MLP hyperparameters

Retrain-to-Predict Workflow

- | | | |
|---|-------------------------|---|
| 1 | Place new training data | Place training Excel in input/ directory |
| 2 | Run training command | <code>python main.py --mode train --input "input/new_data.xlsx"</code> |
| 3 | Pipeline executes | Reads Excel → engineers 6 features → log-transforms targets → fits StandardScalers → trains MLP |
| 4 | Artifacts saved | Saves <code>model.pkl</code> , <code>input_scaler.pkl</code> , <code>target_scaler.pkl</code> to <code>models/</code> |
| 5 | Place prediction data | Place prediction Excel in input/ directory |
| 6 | Run prediction command | <code>python main.py --mode predict --input "input/predict_data.xlsx"</code> |
| 7 | Pipeline executes | Reads Excel → engineers 6 features → scales with saved scaler → predicts → inverse-scales → exp-recovers |
| 8 | Output generated | Writes 4 Excel files to <code>data/predictions/</code> and <code>output/</code> |

All 3 model artifacts must come from the same training run. Replacing one without the others will produce incorrect predictions.

Section 2: Significant Wave Height Prediction

6 vessel motion signals → 6 standard deviation features → 1 Hs prediction (ft)
3,888 simulation CSVs | 245 experiments (8 algorithms × scalers)
Production model: Random Forest (100 trees) | $R^2 = 0.9992$, RMSE = 0.4383 ft

Data Overview

- 3,888 simulation CSV files, each representing one sea state
- Sampling rate: 4 Hz (0.25 s interval)
- All data in Imperial units (ft, deg, s)
- Target: Hs (significant wave height) in feet, encoded in filename (e.g., Dir_0_Hs_3.5.csv)
- Physics verification: $H_s = 4 \times \text{std}(\text{Sea Elevation})$ — confirmed across entire dataset
- Split: Train 3,110 (80%), Test 778 (20%), random_state = 42

CSV Column Specification (9 columns per file):

Column	Unit	Used as Feature	Description
Time (sec)	seconds	No	Simulation timestamp
Surge (ft)	feet	Yes	Longitudinal translation displacement
Sway (ft)	feet	Yes	Lateral translation displacement
Roll (deg)	degrees	Yes	Rotation about longitudinal axis
Pitch (deg)	degrees	Yes	Rotation about lateral axis
Surge Acceleration (ft/sec ²)	ft/s ²	Yes	Longitudinal dynamic acceleration
Sway Acceleration (ft/sec ²)	ft/s ²	Yes	Lateral dynamic acceleration
Sea Elevation (ft)	feet	No	Free-surface elevation (target source)
Wave Direction (deg)	degrees	No	Incident wave direction (constant per file)

Ref: significant_wave_height_prediction_pipeline/input/

Feature Engineering: 6 Standard Deviation Features

#	Feature	Source Signal	Unit	Physical Meaning
1	std_surge	Surge (ft)	ft	Longitudinal translation energy
2	std_sway	Sway (ft)	ft	Lateral translation energy
3	std_roll	Roll (deg)	deg	Roll rotational energy
4	std_pitch	Pitch (deg)	deg	Pitch rotational energy
5	std_surge_acc	Surge Accel (ft/s ²)	ft/s ²	Dynamic longitudinal acceleration energy
6	std_sway_acc	Sway Accel (ft/s ²)	ft/s ²	Dynamic lateral acceleration energy

- Standard deviation captures signal energy: for zero-mean oscillatory signals, std is proportional to RMS amplitude
- Physical basis: higher waves produce larger vessel motions across all 6 degrees of freedom
- Each feature represents the energy content of one motion channel during the simulation window
- 6 features from 6 signals — no derived combinations, no cross-signal features
- No feature scaling applied — the model handles raw feature values directly

Model Selection: 245 Experiments

Full combinatorial grid search: 8 algorithms × X-scalers × Y-scalers = 245 total experiments

8 Algorithms Tested:

- Linear Regression, Ridge, Lasso, SVR
- Decision Tree, Random Forest, Gradient Boosting, KNN

7 X-Scalers:

- None, StandardScaler, MinMaxScaler
- RobustScaler, MaxAbsScaler
- PowerTransformer, QuantileTransformer

5 Y-Scalers:

- None, StandardScaler, MinMaxScaler
- RobustScaler, PowerTransformer

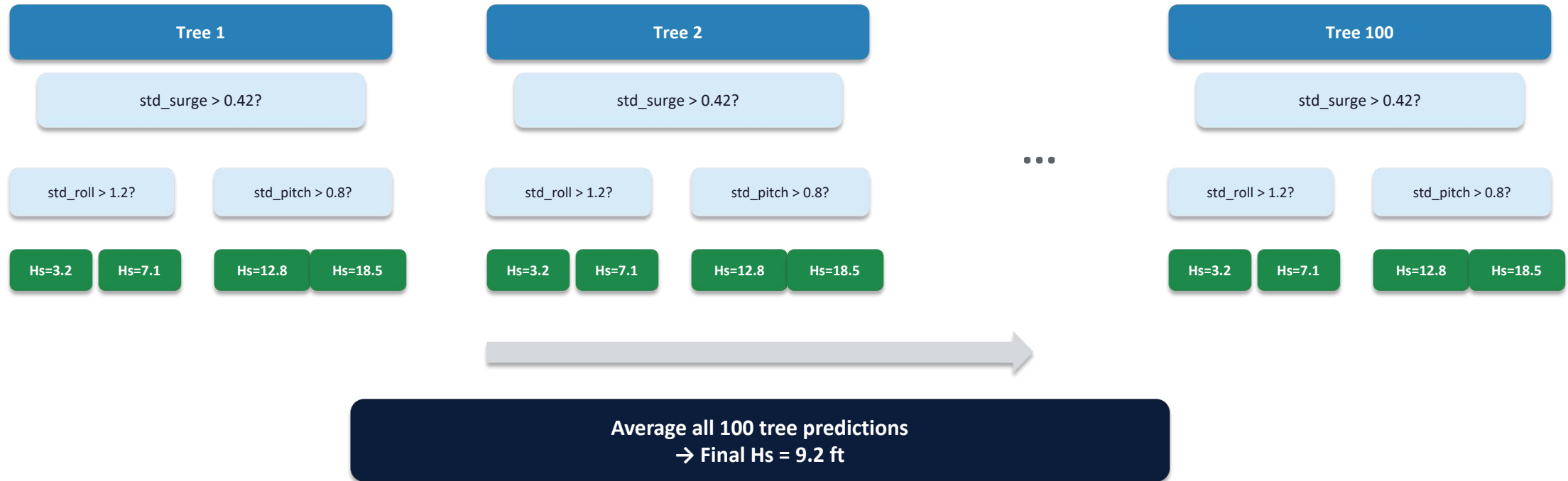
Result: Random Forest with no X-scaler and no Y-scaler achieved the best performance

Best Model	X-Scaler	Y-Scaler	Test MAE (ft)	Test RMSE (ft)	Test R ²
Random Forest	None	None	0.2574	0.4383	0.9992

- Random Forest performed best with no scaling applied to either inputs or targets
- This is expected for tree-based models: decision trees split on rank ordering of feature values, not on absolute magnitudes — scaling does not change rank order and therefore does not affect splits
- Simpler pipeline: no scaler artifacts to maintain, fewer preprocessing steps at prediction time

How Random Forest Works

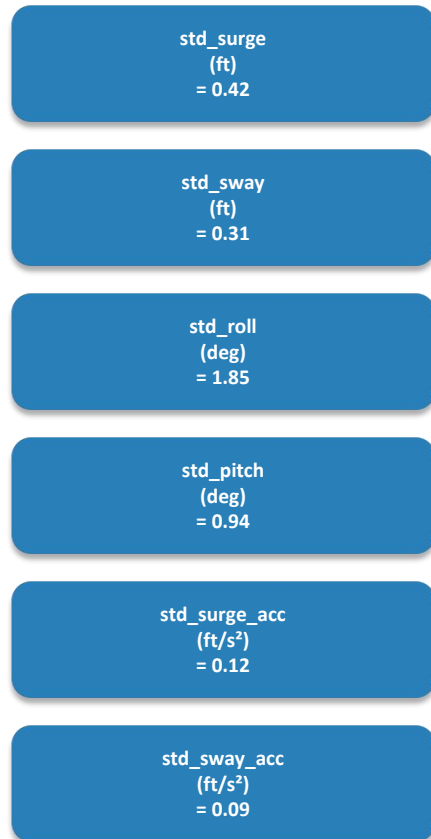
Random Forest: an ensemble of independent decision trees that vote on the prediction



- Each tree sees a random subset of features at each split — decorrelates the trees
- Each tree is trained on a bootstrap sample (random sampling with replacement)
- Individual trees overfit, but averaging 100 trees cancels out the noise
- No scaling needed: trees split on feature value thresholds (rank-based), not magnitudes

Model Architecture: Random Forest (100 Trees)

6 Input Features



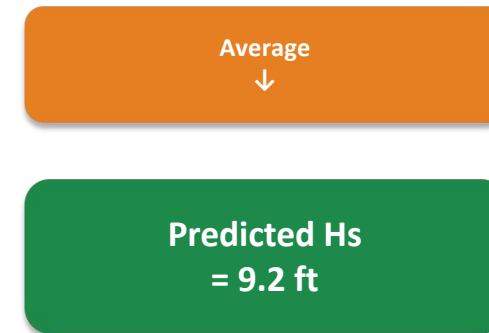
Random Forest 100 Trees



... (100 total)



Output



Configuration: n_estimators=100, random_state=42 | No X-scaler, No Y-scaler | Direct feature values → tree splits → averaged prediction

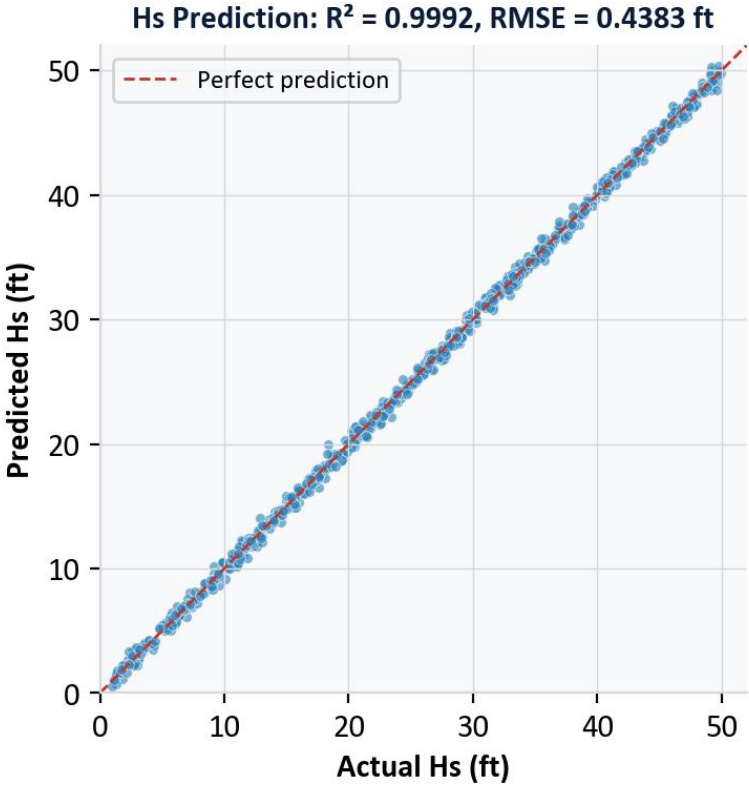
Ref: *significant_wave_height_prediction_pipeline/configs/model_config.json*

Model Performance

Partition	Samples	MAE (ft)	RMSE (ft)	R ²
Train	3,110	0.0593	0.1090	0.9999
Test	778	0.2574	0.4383	0.9992

Test R²
0.9992

Test RMSE
0.4383 ft



- R² = 0.9992: the model explains 99.92% of the variance in Hs
- RMSE = 0.4383 ft: average prediction error is less than 0.5 ft
- MAE = 0.2574 ft on test: median predictions are within ~3 inches
- Train R² = 0.9999: near-perfect fit expected for Random Forest with min_samples_leaf=1 (each leaf can contain a single sample)
- The gap between train and test error (0.06 → 0.26 ft MAE) is small, indicating the model generalizes well without overfitting

Pipeline Flow

Training:



Prediction:



- No scaler artifacts — the simplest pipeline of all three systems
- Training: CSV folder → extract 6 STD features + parse Hs from filename → fit Random Forest → save model
- Prediction: CSV file(s) → extract same 6 features → load model → predict → write Excel report
- Single artifact: models/best_model.pkl (Random Forest with 100 trees)

Production Configuration

Commands:

```
Train:          python main.py --mode train --input "path/to/simulation_csvs/"
Predict (single): python main.py --mode predict --input "path/to/sample.csv"
Predict (batch): python main.py --mode predict --input "path/to/folder/"
```

Production Artifact:

File	Description
models/best_model.pkl	Trained Random Forest (100 trees, random_state=42)

Model Configuration (configs/model_config.json):

Parameter	Value
model_type	RandomForestRegressor
n_estimators	100
random_state	42

Source Code:

- src/features.py — Feature extraction (6 STD features from time-series)
- src/trainer.py — Training pipeline
- src/predictor.py — Inference pipeline
- configs/feature_set.json — Feature definitions and column mappings
- configs/model_config.json — Random Forest hyperparameters

Output: data/predictions/Inference_Final_Report.xlsx — predicted Hs with features

Retrain-to-Predict Workflow

- 1 Place training CSVs** Place simulation CSV files in a folder (filenames encode Hs: Dir_0_Hs_3.5.csv)
- 2 Run training**

```
python main.py --mode train --input "path/to/csv_folder/"
```
- 3 Pipeline trains** Extracts 6 STD features from each CSV → parses Hs from filename → fits Random Forest
- 4 Artifact saved** Saves models/best_model.pkl and data/processed/training_features_extracted.csv
- 5 Place prediction data** Place new CSV file(s) in a folder or provide single file path
- 6 Run prediction**

```
python main.py --mode predict --input "path/to/new_data.csv"
```
- 7 Pipeline predicts** Extracts 6 STD features → loads saved model → predicts Hs (ft)
- 8 Output generated** Writes data/predictions/Inference_Final_Report.xlsx

Single artifact pipeline: only best_model.pkl needs to be managed. No scaler files, no target transformation.

Section 3a: Wave Heading — Circular Regression (0°–360°)

7 motion signals → 156 candidate features → 22 selected via RFE → ExtraTreesRegressor → heading (°)
3,888 CSVs | 166 RFE rounds (~1.08M model fits) | Round 146 → 22 features
Test: CMAE = 0.623°, Max Error = 21.31°, 98.71% within 10°

The Circular Error Problem

Why standard regression fails on angular targets:

- True heading = 1°, Predicted = 359° → Standard error = $|1 - 359| = 358^\circ$, but real angular distance = 2°
- True heading = 0°, Predicted = 360° → Standard error = 360°, but they are the same direction
- Any regression loss (MSE, MAE) using raw angles will heavily penalize predictions near the 0°/360° boundary
- The model learns to avoid predicting near 0° or 360°, creating a systematic bias in that region

Circular Error Metrics:

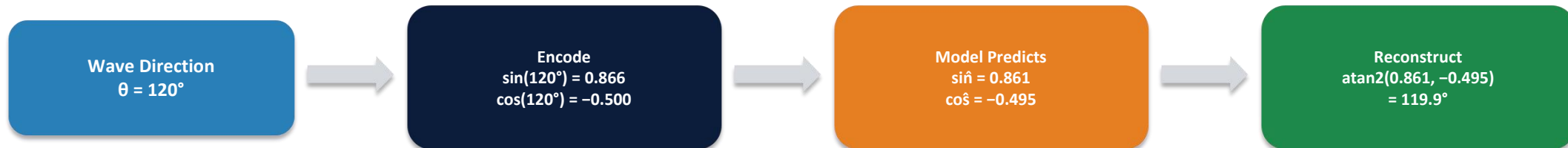
Metric	Formula	Interpretation
Circular Error	$e = \min(\Delta\theta , 360^\circ - \Delta\theta)$ where $\Delta\theta = \text{pred} - \text{true}$	Shortest angular distance between prediction and truth
CMAE	$(1/N) \times \sum e_i$	Circular Mean Absolute Error — average angular error
CRMSE	$\sqrt{(1/N) \times \sum e_i^2}$	Circular Root Mean Square Error
P95	95th percentile of $\{e_1, e_2, \dots, e_n\}$	95% of predictions fall within this error
Max Error	$\max(e_1, e_2, \dots, e_n)$	Worst single prediction — PRIMARY METRIC for wave heading

Max error is the primary focus: even one large angular error could mean the model places the wave on the wrong side of the riser, leading to incorrect engineering decisions.

Sin/Cos Target Encoding

Solution: encode the target angle as two continuous components

- Instead of predicting θ directly, predict $[\sin(\theta), \cos(\theta)]$
- Both sin and cos are smooth, continuous functions with no discontinuity at $0^\circ/360^\circ$
- $\sin(1^\circ) \approx 0.0175$ and $\sin(359^\circ) \approx -0.0175$ — close values for close angles
- Multi-output regression: the model predicts 2 values simultaneously
- Reconstruction: $\theta = \text{atan2}(\sin, \cos)$, mapped to $[0^\circ, 360^\circ)$
- This eliminates the boundary problem entirely



Error = $|120.0^\circ - 119.9^\circ| = 0.1^\circ$ — no boundary artifacts

Data & Feature Extraction Overview

- 3,888 CSV files — same format as Hs pipeline (9 columns, 4 Hz, Imperial units)
- Unit conversion: all linear amplitude columns $\times 0.3048$ (ft $\rightarrow$ m) before feature computation
- 7 input signals: Surge, Sway, Roll, Pitch, Surge Acceleration, Sway Acceleration, Sea Elevation
- Split: Train 3,110 (80%), Validation 389 (10%), Test 389 (10%), stratified on heading $\times$ Hs, seed=456

156 candidate features computed across 6 physical groups:

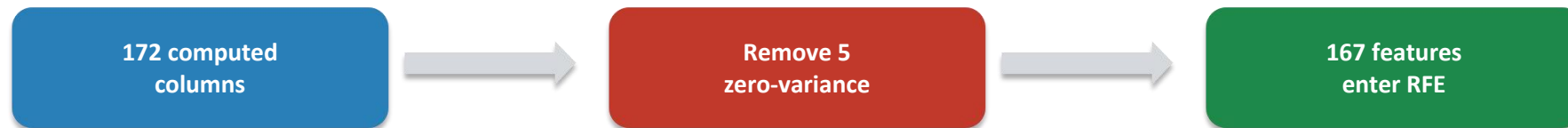
Group	Count	Description
Velocity (6 sub-groups)	56	Surge/Sway velocity kinematics, time statistics, energy ratios, phase, spectra, PCA/ellipse
Position & Acceleration	61	Position/acceleration statistics, cross-correlations, spectral analysis, ellipse geometry
Extended Position	12	Higher-order statistics (skewness, kurtosis), wavelet decomposition
Sea Elevation	8	Wave height (Hs), peak period (Tp), spectral bandwidth, shape parameters
System Linking	16	Phase relationships between vessel motions and sea elevation
Composite	3	Cross-group ratios combining velocity, position, and sea features

Total: 156 features $\rightarrow$ 172 columns after angle encoding (sin/cos pairs for angular features)

Ref: `circular_regression_full_heading/src/features.py` (1,593 Lines)

Zero-Variance Feature Removal

- 172 total computed columns (156 features + additional sin/cos encoded variants)
- 5 features had zero variance — identical value across all 3,888 samples
- These were removed before RFE begins → 167 features remain
- RFE would innately eliminate these features regardless, because zero-variance features carry no information for any tree split and contribute nothing to prediction accuracy
- Removing them upfront saves computation: each RFE round trains a 600-tree ensemble, and 166 rounds × 5 useless features = unnecessary computation that can be avoided
- After removal: 167 features enter the RFE process



RFE Methodology: Why 3 Partitions?

Naive Approach (Train/Test only):

- Train on training set, evaluate on test set, remove worst feature
- Problem: features are selected to look good on the specific test partition
- This is information leakage — test set indirectly influences the model
- Features optimized for one specific data split, not for general signal
- Result appears good on test but may fail on truly unseen data

Our Approach (Train/Validation/Test):

- 3 partitions: Train (fit model), Validation (make drop decisions), Test (report only)
- Features dropped based on validation performance only
- Test set is NEVER consulted for any decision — only for final reporting
- Result: selected features generalize to data the model has never seen
- No information leakage from test partition into feature selection

How this reduces bias and improves generalization:

- Bias reduction: the model cannot overfit to features that happen to correlate with test targets. Each feature must prove its value on the independent validation partition.
- Generalization: fewer features means less opportunity to memorize noise. Each remaining feature must independently contribute genuine physical signal, not statistical artifacts of one data partition.
- Evidence of success: validation CMAE = 0.438° → test CMAE = 0.623° (small gap = good generalization). If test had been used in decisions, we would expect artificially good test results that do not hold on new data.

RFE Process: 166 Rounds, ~1.08M Model Fits

- Starting point: 167 features (after zero-variance removal)
- 166 rounds of elimination: each round removes 1 feature → 167 down to 2 features
- Each round: retrain ExtraTreesRegressor (600 trees) → evaluate on validation set
- Decision metric: max_error_deg on validation set
- P95 criterion: a feature is dropped only if the validation P95 error does not degrade significantly
- Test set plotted for monitoring but NEVER used in any drop decision
- Total model fits across 166 rounds: ~1.08 million (600 trees × 166 rounds × evaluations)
- Optimal round selected: Round 146 → 22 features

Starting Features

167

RFE Rounds

166

Model Fits

~1.08M

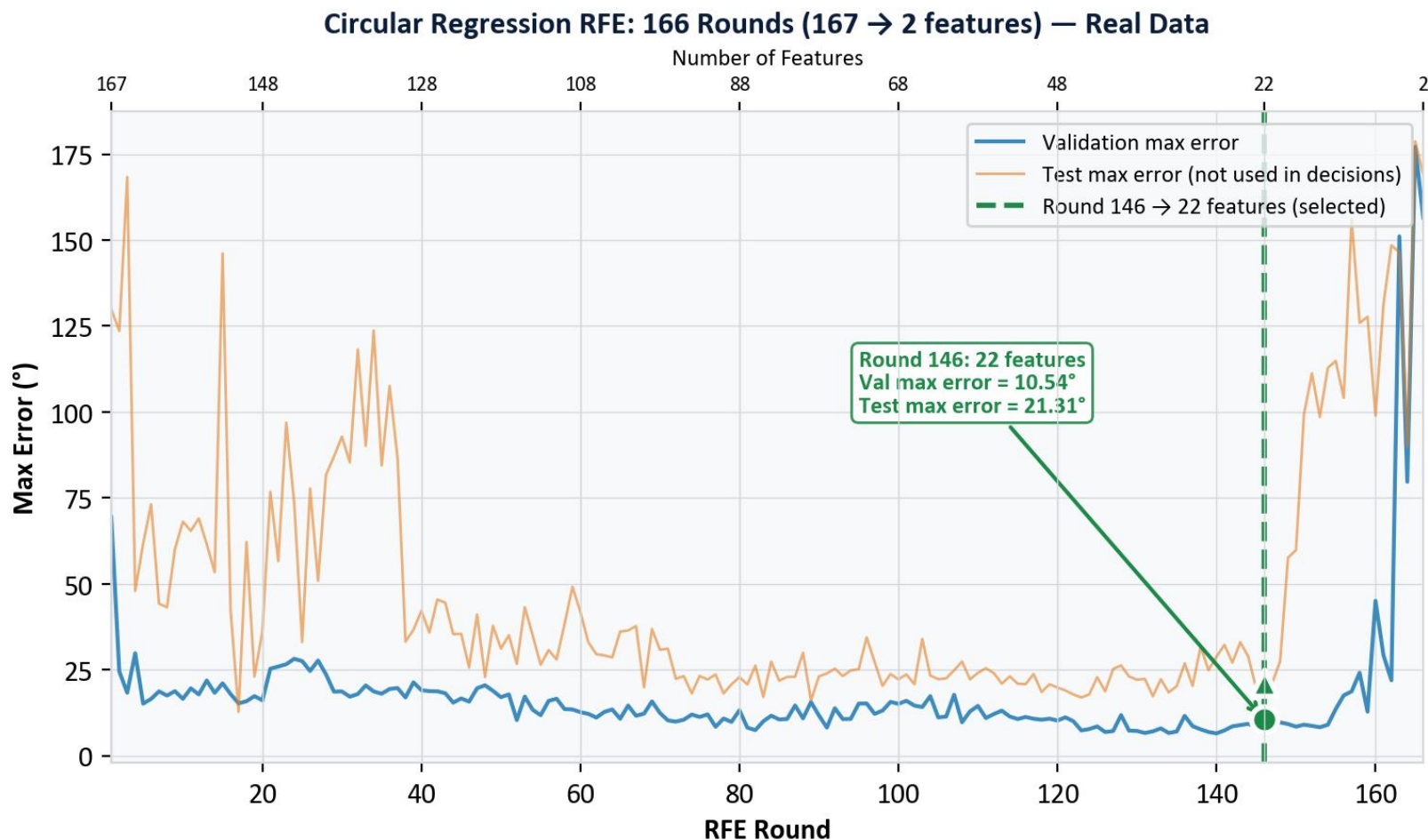
Selected Round

146

Final Features

22

RFE Trajectory: Validation Max Error vs Round



- Round 1 (167 features): val max error = 69.42°
- Round 146 (22 features): val max error = 10.54°
- Round 166 (2 features): val max error = 156.55°
- Error DECREASES as features are removed — the removed features were adding noise, not signal
- After Round 146, removing further features eliminates essential signal and error increases
- Test line shown for reference but was not used in any selection decision

Selected 22 Production Features

#	Feature Name	Group	Physical Meaning
1	vel_diff_sway_surge_time_energy_ratio	Velocity	Sway vs surge kinetic energy fraction
2	vel_diff_hilbert_phase_lag_mean_deg_sin	Velocity	Sin of sway-surge velocity phase lag
3	vel_diff_xspec_phase_lag_waveband_deg_cos	Velocity	Cos of cross-spectral phase in wave band
4	pos_sway_std_m	Position	Lateral position standard deviation (m)
5	acc_surge_std_ms2	Acceleration	Surge acceleration std (m/s²)
6	acc_surge_variance_m2s4	Acceleration	Surge acceleration variance (m²/s⁴)
7	pos_xcorr_roll_pitch_coeff	Position	Roll-pitch cross-correlation coefficient
8	pos_pitch_spectral_total_energy	Position	Total spectral energy in pitch
9	pos_xspec_phase_lag_roll_pitch_deg_sin	Position	Sin of roll-pitch cross-spectral phase
10	pos_xspec_phase_lag_roll_pitch_deg_cos	Position	Cos of roll-pitch cross-spectral phase
11	pos_xspec_coherence_roll_pitch_max	Position	Peak roll-pitch coherence

Ref: circular_regression_full_heading/configs/feature_set.json

Selected 22 Production Features

#	Feature Name	Group	Physical Meaning
12	pos_dominant_motion_dir_deg_cos	Position	Cos of dominant horizontal motion direction
13	pos_roll_pitch_phase_change_low_high_freq_rad	Position	Roll-pitch phase change across wave band
14	pos_sway_surge_xcorr_max_lag_samples	Position	Peak cross-correlation lag (sway-surge)
15	pos_surge_hilbert_envelope_skewness	Position	Surge envelope skewness
16	pos_ellipse_orientation_deg_cos	Position	Cos of motion ellipse orientation
17	sea_elevation_peak_period_Tp_s	Sea Elev.	Dominant wave period (s)
18	link_resonance_proximity_roll_vs_sea_s	Linking	Roll vs sea period proximity (s)
19	link_phase_lag_at_peak_freq_roll_vs_sea_deg_cos	Linking	Cos of roll-sea phase lag at peak freq
20	link_resonance_proximity_surge_vs_sea_s	Linking	Surge vs sea period proximity (s)
21	link_phase_lag_at_peak_freq_surge_vs_sea_deg_sin	Linking	Sin of surge-sea phase lag at peak freq
22	link_phase_lag_at_peak_freq_sway_vs_sea_deg_sin	Linking	Sin of sway-sea phase lag at peak freq

Per-Feature Normalization

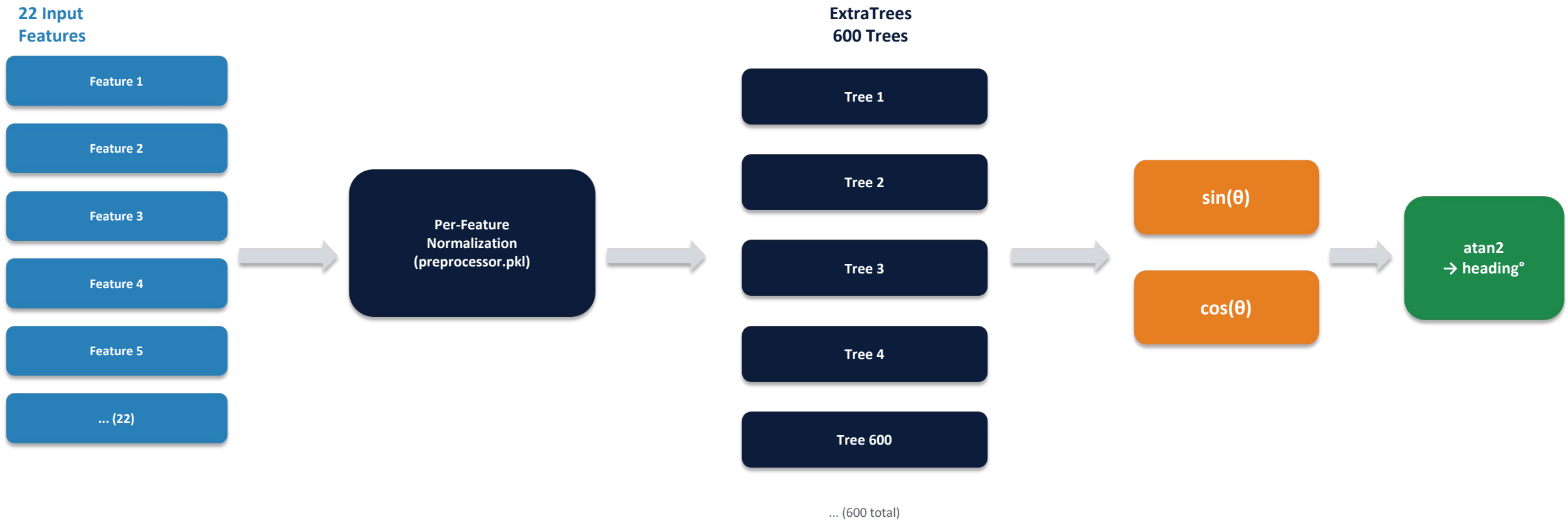
Each of the 22 features has its own transform + scaler combination:

Transform	Description	Applied To
yeo-johnson	Power transform for normalizing skewed distributions	Multiple features
sqrt_signed	Signed square root: $\text{sign}(x) \times \sqrt{ x }$	Selected features
square	x^2 — amplifies large values	Selected features
exp	Exponential — expands compressed ranges	Selected features
none	No transformation applied	Features already well-distributed

Scaler	Description	Applied To
StandardScaler	Z-score: $(x - \text{mean}) / \text{std}$	Multiple features
MinMaxScaler	Scales to [0, 1] range	Selected features
RobustScaler	Uses median and IQR (outlier-resistant)	Selected features
QuantileTransformer	Maps to uniform or normal distribution	Selected features
none	No scaling applied	Features already in model-friendly range

- Unlike the Hs pipeline (no scaling) or fatigue pipeline (single StandardScaler for all features), this pipeline applies per-feature normalization because the 22 features span vastly different distributions
- Phase angles range $[-180^\circ, 180^\circ]$, spectral energies range $[0, \infty)$, correlation coefficients range $[-1, 1]$
- The fitted preprocessor is saved as preprocessor.pkl and must be loaded at prediction time

Model Architecture: ExtraTreesRegressor



ExtraTrees vs Random Forest: ExtraTrees additionally randomizes split thresholds (not just feature subsets), further reducing variance. | `n_estimators=600, max_features=0.5, min_samples_leaf=1, bootstrap=false, seed=456`

5-Fold Cross-Validation Results

5-fold CV on the training set (3,110 samples): each fold trains on $4/5 = 2,488$ samples (64.0% of total 3,888)

Fold	CMAE (°)	CRMSE (°)	Max Error (°)	Acc $\leq 10^\circ$
Fold 1	1.446	10.366	171.27	98.07%
Fold 2	2.632	15.359	174.84	96.14%
Fold 3	1.582	11.401	172.44	97.59%
Fold 4	1.165	8.083	150.93	97.59%
Fold 5	1.714	10.402	144.14	97.59%
Mean $\pm$ Std	1.708 $\pm$ 0.497	11.122 $\pm$ 2.381	162.73 $\pm$ 12.64	97.40% $\pm$ 0.65%

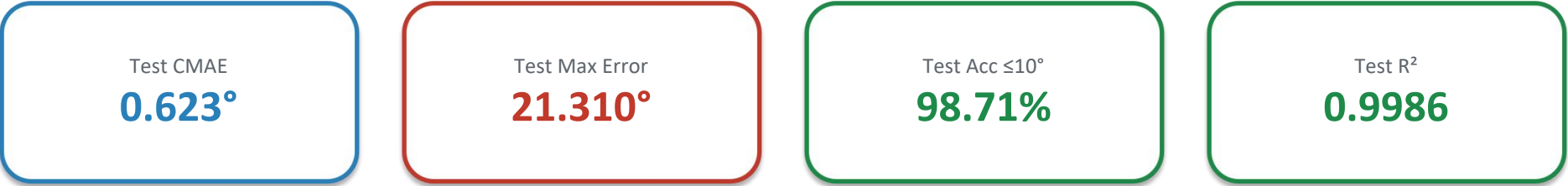
Why CV performance is lower than final model performance:

- Each CV fold trains on 2,488 samples (64.0% of total 3,888) instead of 3,110 samples (80.0%)
- With 622 fewer training samples per fold, the model has less data to learn the full angular range
- CV max errors reach 144° – 175° because some folds lack sufficient samples in certain heading angles
- The final model trains on all 3,110 samples AND uses 22 RFE-selected features — both factors contribute to much better performance
- CV confirms the model generalizes (mean CMAE = 1.71° , mean acc $\leq 10^\circ$ = 97.40%) even with reduced training data

Validation & Test Performance (Primary: Max Error)

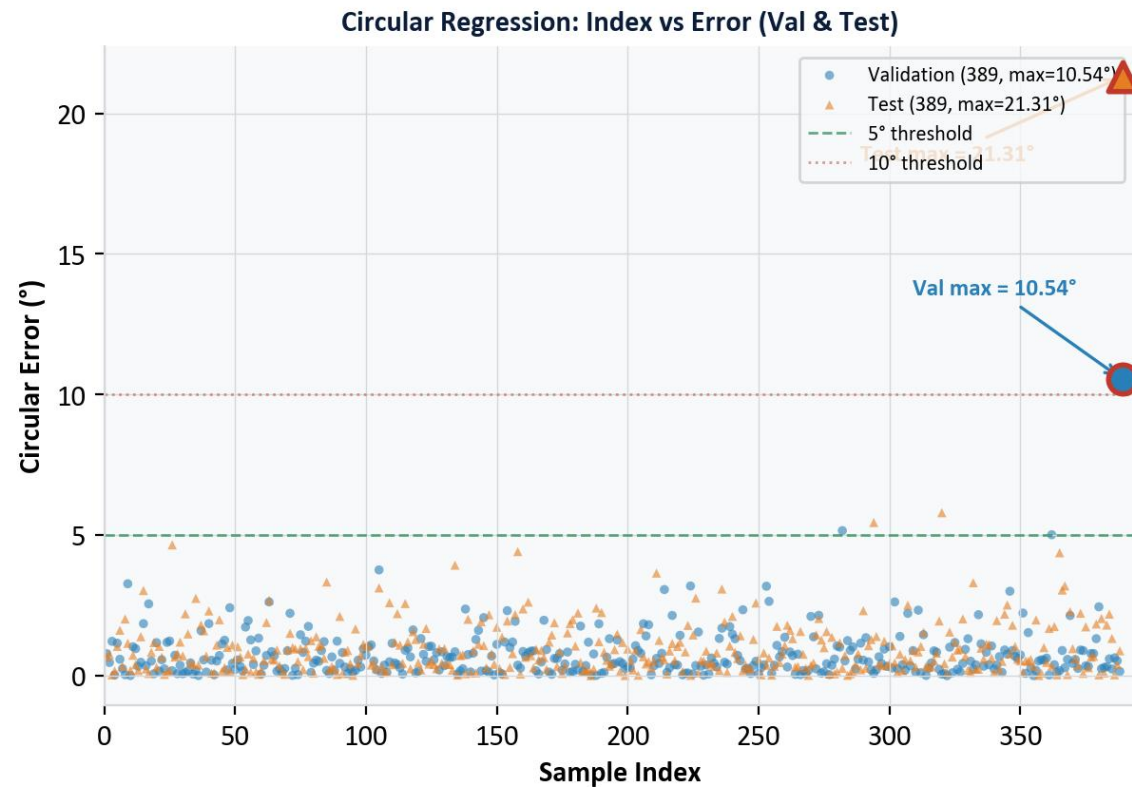
Final model trained on 3,110 samples (80% of total 3,888) with 22 RFE-selected features:

Split	Samples	CMAE (°)	CRMSE (°)	Max Error (°)	Acc ≤5°	Acc ≤10°	R²
Validation	389	0.438	1.265	10.536	97.69%	99.74%	0.9995
Test	389	0.623	2.163	21.310	95.89%	98.71%	0.9986



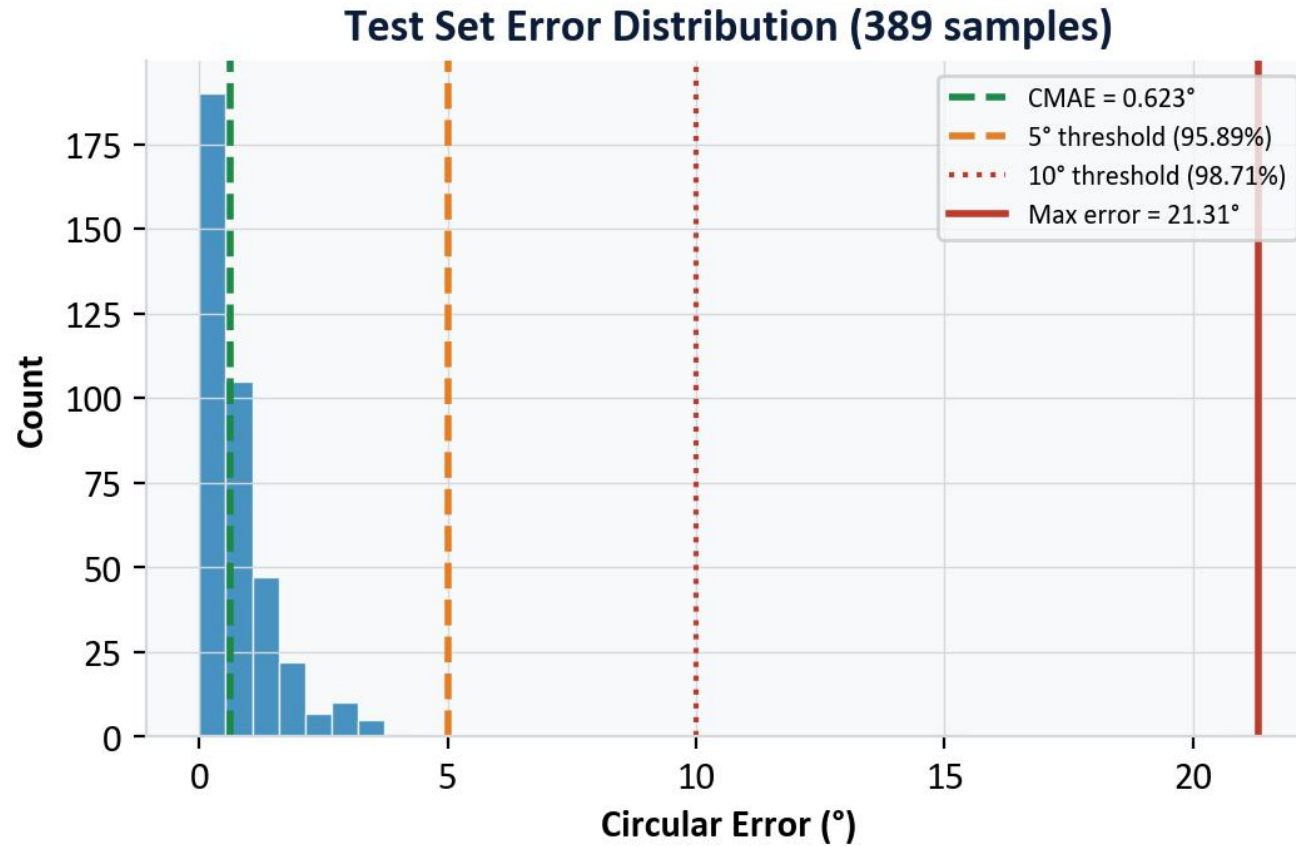
- Max error analysis: validation worst case = 10.54°, test worst case = 21.31°
- 98.71% of test predictions are within 10° of truth, 95.89% within 5°
- Generalization gap: val CMAE 0.438° → test CMAE 0.623° — small gap confirms features generalize well
- Train error is ~0° because min_samples_leaf=1 allows exact fit — expected for tree ensembles, not overfitting

Predicted vs Actual Heading



- Points cluster tightly along the diagonal
- Color intensity shows error magnitude
- $\pm 10^\circ$ band captures 98.71% of predictions
- Worst prediction: 21.31° off
- No systematic bias at $0^\circ/360^\circ$ boundary — sin/cos encoding eliminated the discontinuity
- Predictions are distributed uniformly across all heading angles (0° to 360°)

Test Set Error Distribution



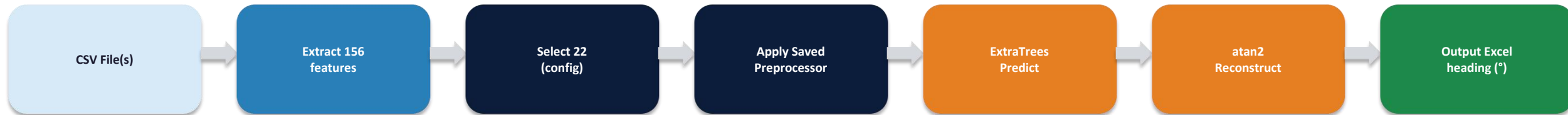
- Most errors concentrated near 0°
- CMAE = 0.623° (green line)
- 95.89% of errors below 5°
- 98.71% of errors below 10°
- Max error = 21.31° (single outlier)
- Distribution is heavily right-skewed — the typical error is sub-degree

Pipeline Flow

Training:



Prediction:



- Both pipelines use the same features.py (1,593 lines) for feature extraction
- Preprocessor normalizes features to the same distribution the model was trained on
- atan2 reconstruction maps [sin, cos] back to heading in degrees [0°, 360°)

Production Configuration

Commands:

```
Train:          python main.py --mode train --input "path/to/simulation_data/"
Predict (single): python main.py --mode predict --input "path/to/sample.csv"
Predict (batch): python main.py --mode predict --input "path/to/folder/"
```

Production Artifacts (both from same training run):

File	Description
models/ensemble_model.pkl	ExtraTreesRegressor (600 trees, 22 features → 2 outputs)
models/preprocessor.pkl	Fitted per-feature normalization pipeline

Model Configuration (configs/model_config.json):

Parameter	Value
model_type	ExtraTreesRegressor
n_estimators	600
max_features	0.5
min_samples_leaf	1
min_samples_split	2
bootstrap	false
random_state	456

Output Files:

- Inference_Final_Report.xlsx — predicted heading + raw features
- Inference_Raw_Features.xlsx — 22 features before normalization
- Inference_Transformed_Features.xlsx — 22 features after normalization

Source Code:

- src/features.py (1,593 lines) — All 156 feature computations
- src/trainer.py — Training pipeline + preprocessor fitting
- src/predictor.py — Inference pipeline

Ref: circular_regression_full_heading/configs/

Section 3b: Wave Heading — Hierarchical Quadrant Cascade

Stage 1: ExtraTrees Classifier (36 features → 4 quadrants, 100% test accuracy)

Stage 2: Per-quadrant regressors (21 features → sin/cos → heading°)

171 classification RFE rounds | 36,752 regression experiments | Max error = 12.50° (Q4)

Approach Overview: Two-Stage Cascade

Hypothesis: if we first identify the correct 90° quadrant, a specialized regressor can achieve lower max error within that narrower range.

Stage 1: Classification

- ExtraTrees Classifier (500 trees, 36 features)
- Classifies into Q1 (0°–90°), Q2 (90°–180°), Q3 (180°–270°), Q4 (270°–360°)
- Test accuracy: 100.0% (584 samples, 146 per quadrant)

Stage 2: Per-Quadrant Regression

- 4 specialized regressors (one per quadrant, 21 features each)
- Each predicts sin/cos within its 90° range
- XGBoost for Q1–Q3, Decision Tree for Q4

Combined prediction pipeline:



Data split: Train 2,720 (70%), Validation 584 (15%, 146/quadrant), Test 584 (15%, 146/quadrant), seed=42

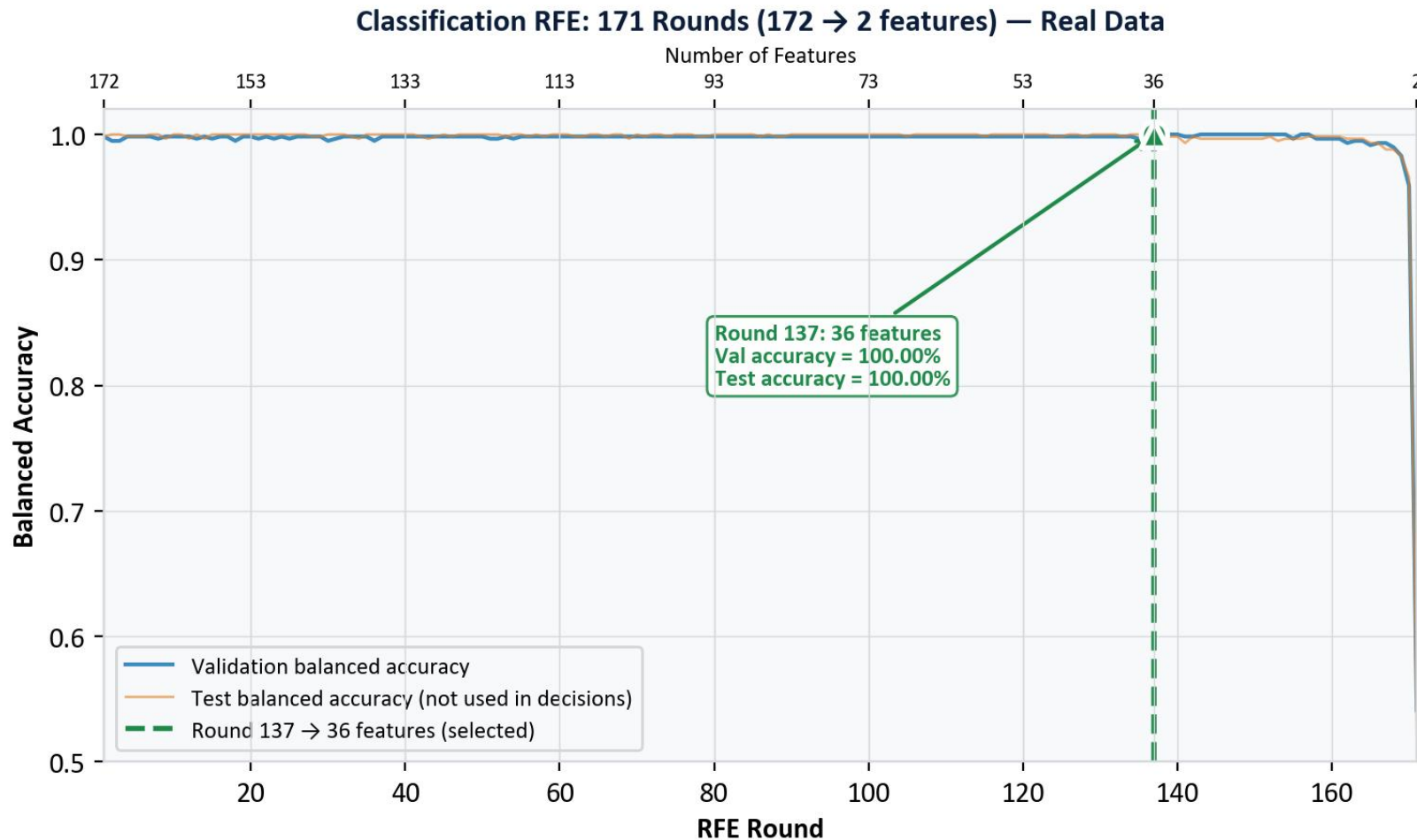
Classification RFE: Starting at 172 Features

- Same 172 features computed as in circular regression (156 base + encoded variants)
- RFE starts at 172 features — zero-variance features are NOT pre-removed
- The RFE elimination process innately removes zero-variance features because they carry zero information for any classification split — a feature with the same value across all samples cannot separate any class from another
- 171 rounds of elimination: 172 down to 1 feature
- Decision metric: balanced_accuracy on validation set (584 samples, 146 per quadrant)
- Test set NEVER used in any drop decision — only reported for monitoring
- Each round: retrain ExtraTreesClassifier (500 trees), 5-fold CV, 324 hyperparameter configurations
- Optimal round: Round 137 → 36 features

Contrast with circular regression: circular RFE pre-removed 5 zero-variance features (172 → 167) to save computation. Classification RFE starts at 172 and lets the elimination demonstrate that these features are naturally dropped. Both approaches reach the same outcome.



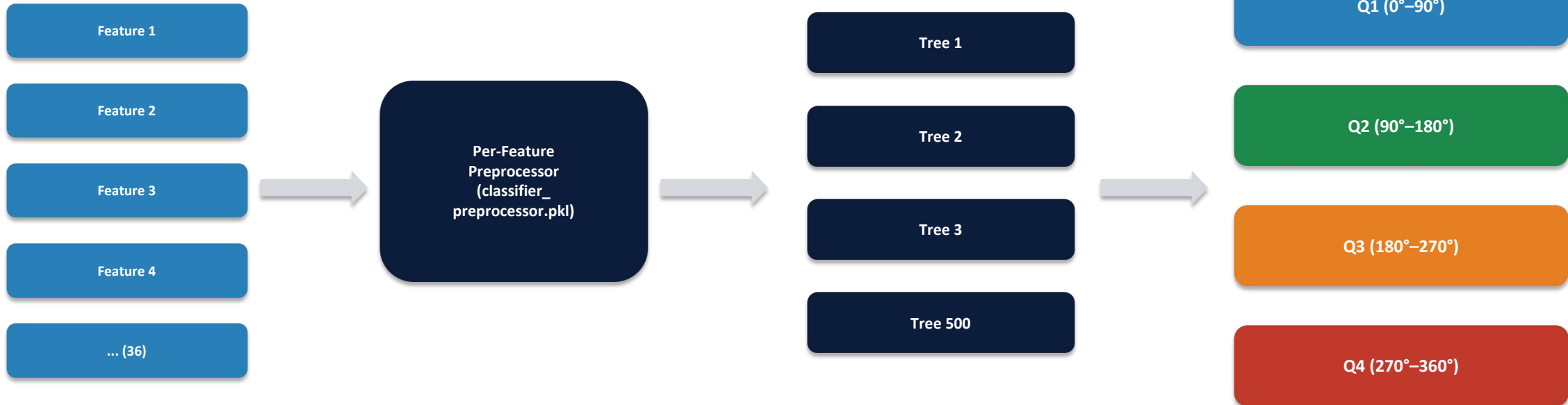
Classification RFE Trajectory



- Round 1 (172 features): val = 0.9932
- Round 137 (36 features): val = 1.0000
- Round 171 (1 feature): val = 0.5411
- Accuracy INCREASES as noisy features are removed — peaks at Round 137
- After Round 137, removing essential features causes accuracy to drop sharply
- Round 137 drops:
link_coherence_at_peak_freq_sway_vs_sea_ratio
- Val improved from 0.9983 to 1.0000 after dropping this feature

Classification Model: ExtraTrees (500 Trees)

36 Input
Features



Config: n_estimators=500, max_features='sqrt', random_state=42 | 5-fold CV (324 configs): best_cv_mean = 0.9956 ± 0.0025

Ref: *hierarchical_quadrant_cascaded_pipeline/configs/models_config.json*

Classification Performance

Validation Set (584 samples, 146 per quadrant):

Metric	Value
Accuracy	99.83%
Balanced Accuracy	99.83%
Cohen's Kappa	0.9977
MCC	0.9977

Per-Class Precision	Per-Class Recall	Per-Class F1
Q1: 1.000	Q1: 1.000	Q1: 1.000
Q2: 0.993	Q2: 1.000	Q2: 0.997
Q3: 1.000	Q3: 1.000	Q3: 1.000
Q4: 1.000	Q4: 0.993	Q4: 0.997

1 misclassification in Q4 (145/146 correct). All other quadrants perfect.

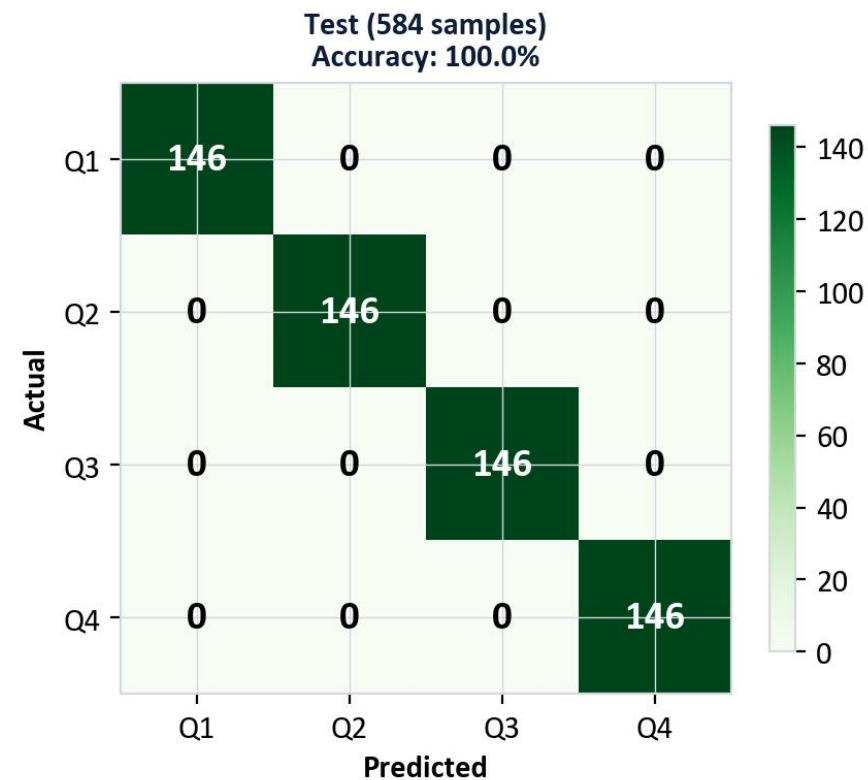
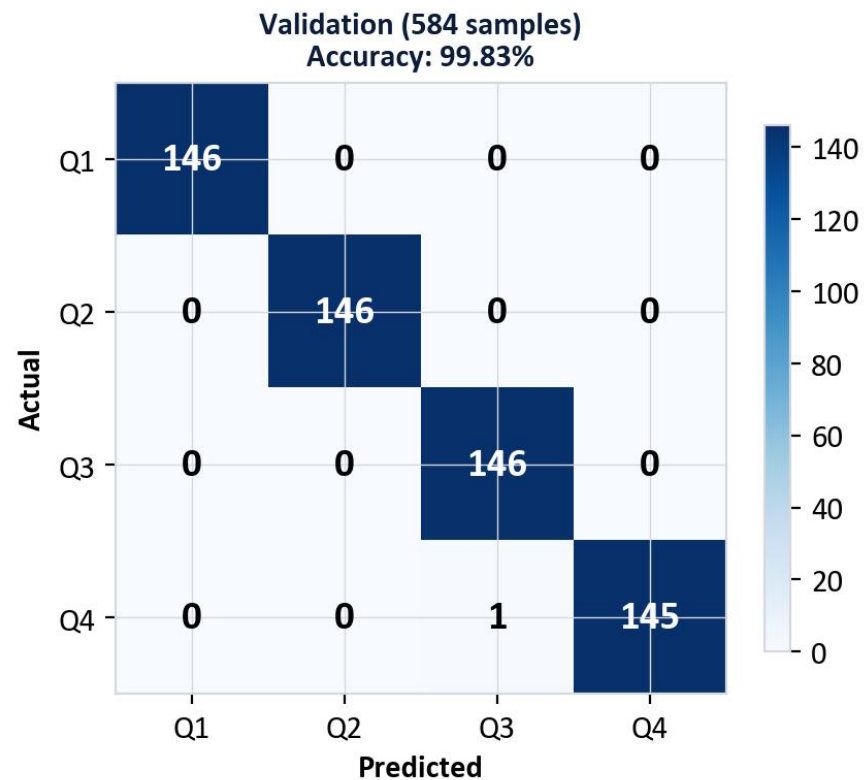
Test Set (584 samples, 146 per quadrant):

Metric	Value
Accuracy	100.0%
Balanced Accuracy	100.0%
Cohen's Kappa	1.000
MCC	1.000

Per-Class Precision	Per-Class Recall	Per-Class F1
Q1: 1.000	Q1: 1.000	Q1: 1.000
Q2: 1.000	Q2: 1.000	Q2: 1.000
Q3: 1.000	Q3: 1.000	Q3: 1.000
Q4: 1.000	Q4: 1.000	Q4: 1.000

0 misclassifications. Perfect classification on all 584 test samples. Boundary accuracy within 2° = 100%, within 5° = 100%, within 10° = 100%.

Confusion Matrices



- Validation: 583/584 correct (99.83%), 1 Q4 sample misclassified into Q3
- Test: 584/584 correct (100.0%), perfect diagonal — zero misclassifications
- Even samples near quadrant boundaries (within 2° of 0°, 90°, 180°, 270°) are correctly classified
- Confidence calibration: mean confidence = 96.7%, ECE (10-bin) = 3.3% (well-calibrated)

Regression Stage: 36,752 Experiments

4 target encodings × 4 quadrants × 2,297 configs = 36,752 total regression experiments

#	Dataset	Target Encoding	Description
1	raw_angle	Direct angle (°)	Predict the angle value directly within 90° range
2	transformed_raw_angle	Transformed angle	Apply feature transforms before angle prediction
3	raw_sincos	sin/cos encoding	Predict [sin(θ), cos(θ)] from raw features
4	transformed_sincos	Transformed sin/cos	Apply transforms then predict sin/cos

- 2,297 configurations per quadrant per dataset: includes XGBoost, Decision Tree, Random Forest, KNN, and other algorithms with hyperparameter sweeps
- Training samples per quadrant: 777 (from 2,720 total training set)
- Test samples per quadrant: 195 (from 584 total test set)
- Each configuration trained and evaluated independently — no cross-quadrant optimization



Dataset Comparison: Why Dataset 3 (raw_sincos) Was Selected

Best max error achieved per quadrant across all 4 target encoding datasets:

Dataset	Q1 Max (°)	Q2 Max (°)	Q3 Max (°)	Q4 Max (°)	Q1 CMAE (°)	Q2 CMAE (°)	Q3 CMAE (°)	Q4 CMAE (°)
D1: raw_angle	3.86	5.00	6.85	10.00	0.31	0.36	0.38	0.86
D2: transformed_angle	8.30	6.08	7.44	4.24	0.71	0.57	0.59	0.54
D3: raw_sincos	4.36	4.01	4.48	12.50	0.29	0.29	0.25	0.50
D4: transformed_sincos	5.99	5.34	7.45	29.17	0.70	0.51	0.51	3.24

Selection logic — Dataset 3 is the only dataset where both CMAE and max error are simultaneously low across Q1–Q3:

- D3 Q1–Q3: CMAE = 0.25–0.29° AND max error = 4.01–4.48° — no other dataset achieves sub-0.3° CMAE and sub-5° max error on three quadrants simultaneously
- D1 wins Q1 (3.86° vs 4.36°), but its CMAE spreads out: Q4 reaches 0.86° CMAE and 10.00° max error. D2 wins Q4 (4.24°), but its Q1 is worst at 8.30°
- Sin/cos encoding helps: comparing D1 vs D3 (both raw features, angle vs sin/cos target), Q2 improves 5.00° → 4.01° and Q3 improves 6.85° → 4.48° — avoids 0°/360° discontinuity
- Raw features outperform transformed: tree-based models are invariant to monotonic transformations, so Box-Cox/Yeo-Johnson adds noise to split decisions without improving accuracy
- Q4's 12.50° is acceptable: it comes from a single outlier — the P95 is 5.00° and median is 0.00°. Q4 is the hardest quadrant because sin/cos encoding has its weakest gradient near 0° (cos → 1, sin → 0), making small angle differences hard to resolve
- D4 fails entirely: Q4 collapses to 29.17° max error with 3.24° CMAE — unusable

Per-Quadrant Regression Models (raw_sincos)

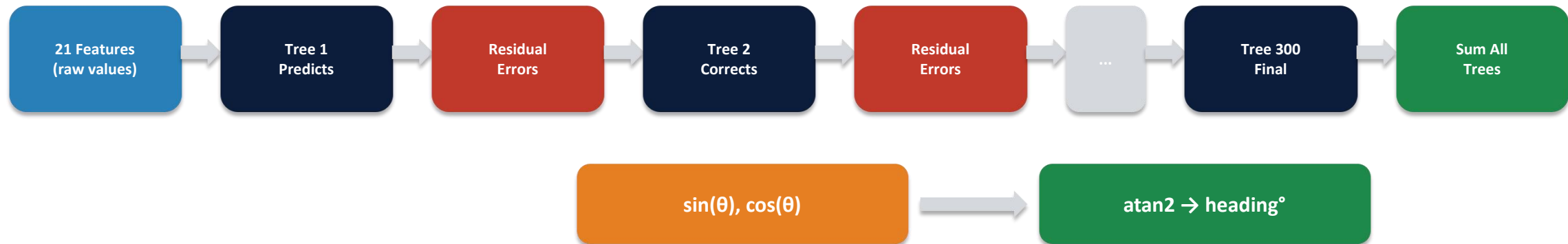
Quadrant	Range	Model	Key Hyperparameters
Q1	0°–90°	XGBoost (300 trees)	colsample=0.8, lr=0.1, depth=7, subsample=0.7
Q2	90°–180°	XGBoost (300 trees)	colsample=0.8, lr=0.1, depth=7, subsample=0.7
Q3	180°–270°	XGBoost (300 trees)	colsample=0.8, lr=0.1, depth=7, subsample=1.0
Q4	270°–360°	Decision Tree	max_features=0.5, min_samples_leaf=2

- Q1–Q3 use XGBoost: gradient boosted trees that sequentially correct prediction errors, 300 trees per quadrant, each tree learns from the residuals of the previous ensemble
- Q3 uses subsample=1.0 (full dataset) while Q1 and Q2 use 0.7 (70% subsample per tree) — Q3 benefits from seeing all training samples at each boosting iteration
- Q4 uses a simpler Decision Tree: Q4 is the most difficult quadrant (max error = 12.50°) and XGBoost did not provide better performance — the Decision Tree achieved the best result from the 2,297 configurations tested for Q4
- All quadrant models predict $[\sin(\theta), \cos(\theta)]$ within the 90° range, then reconstruct with atan2

Training times: XGBoost Q1–Q3 ~0.9 s each | Decision Tree Q4 ~0.01 s | 777 training samples per quadrant, 195 test samples per quadrant

XGBoost Model Visualization (Q1–Q3)

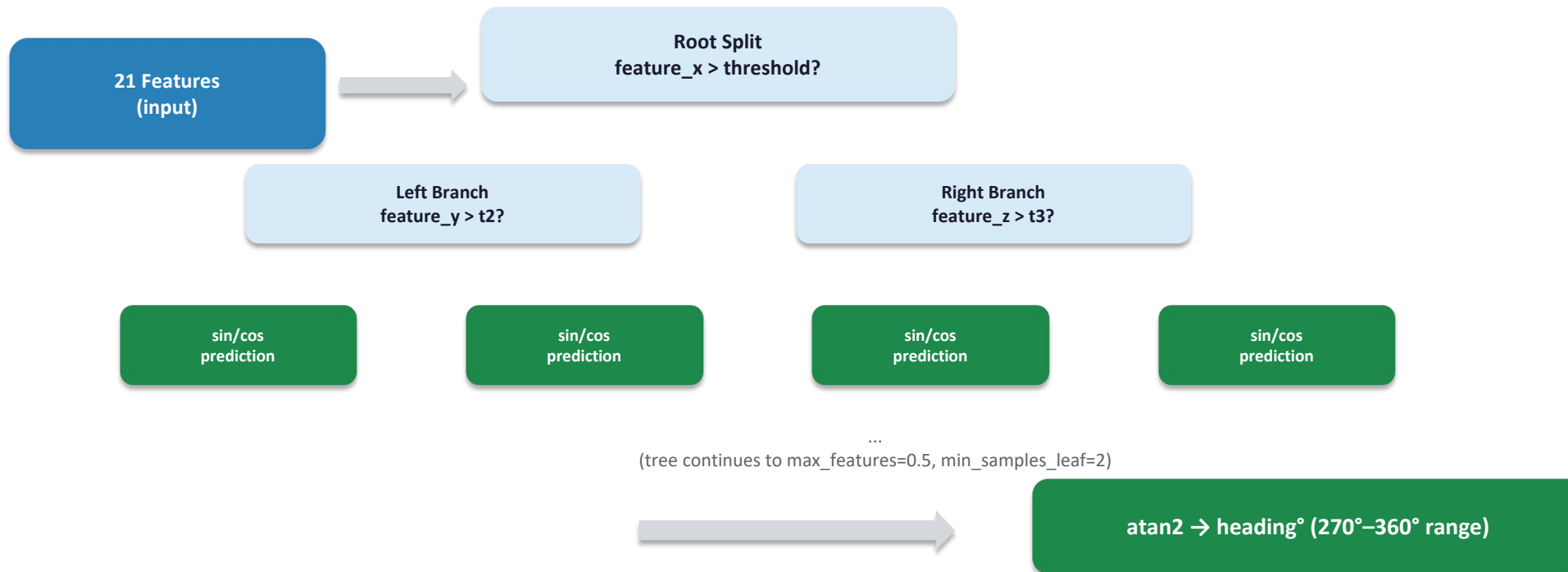
XGBoost: Gradient Boosting — each tree corrects the errors of the previous ensemble



- Boosting (sequential): Tree 1 makes initial predictions, Tree 2 learns from Tree 1's errors, Tree 3 corrects remaining errors, etc. Each tree focuses on the mistakes of the ensemble so far.
- Learning rate = 0.1: each tree's contribution is scaled down by 0.1 to prevent overshooting. 300 trees × 0.1 learning rate = gradual, controlled improvement.
- Max depth = 7: each tree can model interactions between up to 7 features at a time.
- Subsample = 0.7 (Q1/Q2) or 1.0 (Q3): fraction of training data used per tree. Sub-sampling adds stochastic regularization to prevent overfitting.

Decision Tree Model Visualization (Q4)

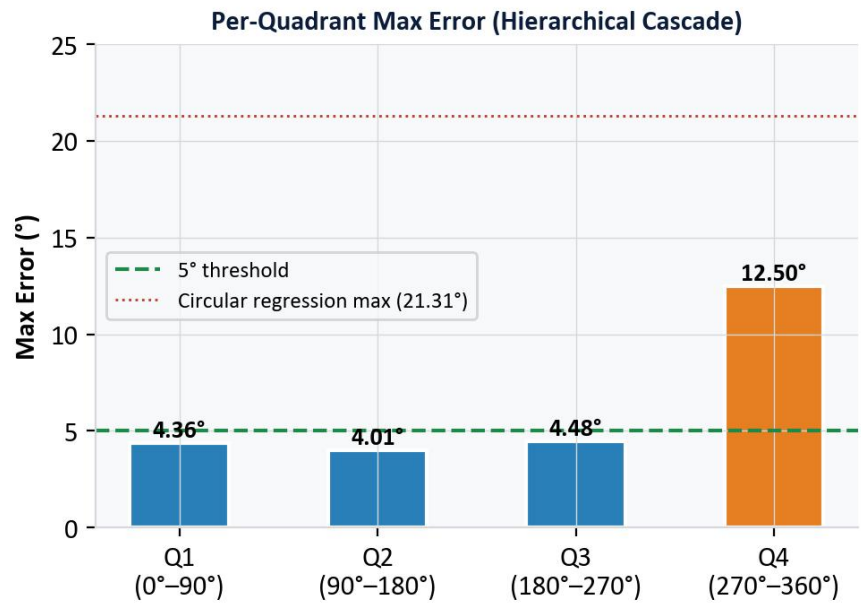
Q4 (270°–360°): Single Decision Tree — the simplest model structure



- Single tree (not an ensemble): no boosting, no bagging — one decision tree makes the prediction
- `max_features=0.5`: at each split, the tree considers 50% of the 21 features (random subset)
- `min_samples_leaf=2`: each leaf must contain at least 2 training samples (prevents overfitting to single points)
- `max_depth=None`: tree grows as deep as needed until leaf constraints are met
- Q4 uses this simpler model because XGBoost did not improve performance for Q4 across the 2,297 configurations tested — the Decision Tree achieved the lowest max error (12.50°)

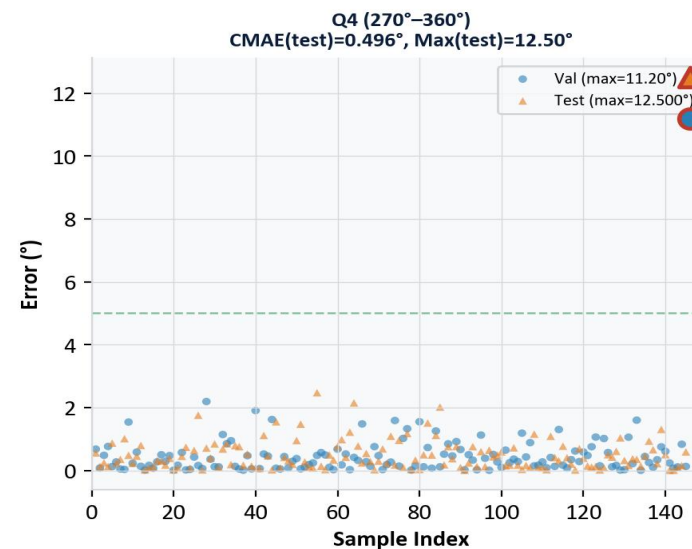
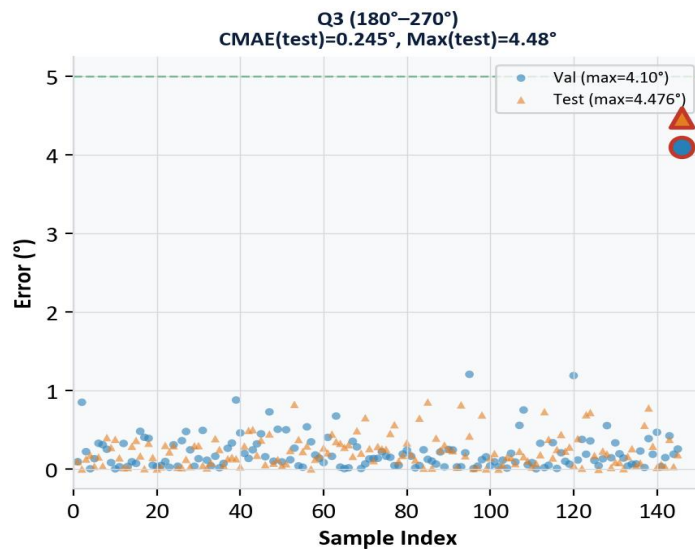
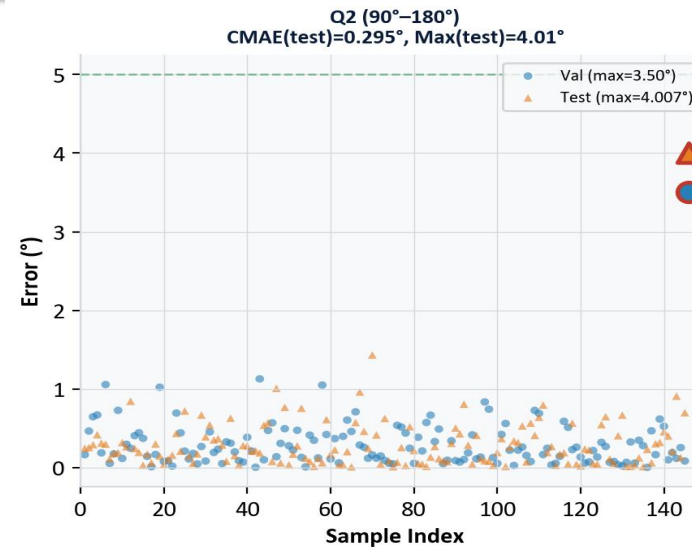
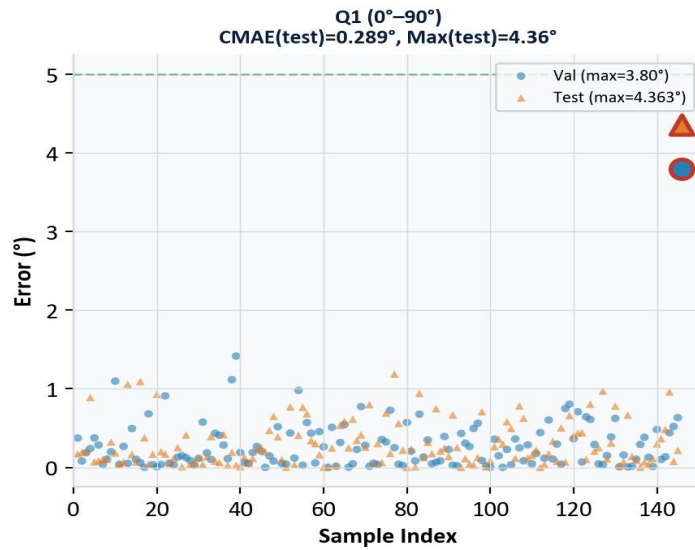
Per-Quadrant Regression Performance (Max Error Focus)

Quadrant	Model	CMAE (°)	CRMSE (°)	P95 (°)	Max Error (°)
Q1 (0°–90°)	XGBoost	0.289	0.793	1.534	4.363
Q2 (90°–180°)	XGBoost	0.295	0.712	1.857	4.007
Q3 (180°–270°)	XGBoost	0.245	0.773	1.437	4.476
Q4 (270°–360°)	Decision Tree	0.496	1.847	5.000	12.500



- Q1–Q3: all below 5° max error, sub-degree CMAE
- Q4: 12.50° max error — the worst quadrant
- Q1–Q3 P95 below 2°: 95% of predictions within 2°
- Q4 P95 = 5.0°: 95% within 5° but tail extends to 12.5°
- Compared to circular regression max = 21.31°, the hierarchical approach reduces worst-case error by 41% (21.31° → 12.50°)

Per-Quadrant: Predicted vs Actual



Hierarchical vs Circular Regression Comparison

Metric	Circular (3a)	Hierarchical (3b)
Approach	Single model, full 0°–360°	Classifier + 4 quadrant regressors
RFE Rounds	166 (167→2)	171 classification (172→1)
Production Features	22	36 (classifier) + 21 (regressors)
Total Experiments	~1.08M model fits	36,752 regression + classification RFE
Production Models	1 model + 1 preprocessor	5 models + 2 preprocessors
CMAE (test)	0.623°	0.245°–0.496° per quadrant
Max Error (test)	21.31°	12.50° (Q4 worst)
Acc ≤10°	98.71%	Q1–Q3: ~100%, Q4: varies

- The hierarchical approach reduces worst-case max error from 21.31° to 12.50° through quadrant specialization
- Trade-off: more complex pipeline (5 models + 2 preprocessors vs 1 model + 1 preprocessor)
- The circular approach is simpler to maintain and deploy, with a single inference path
- The hierarchical approach provides finer angular resolution within each quadrant (sub-5° max for Q1–Q3)
- Both approaches use the same features.py (1,593 lines) for feature extraction

Physics: How Features Identify Wave Direction

Why vessel motion features can discriminate wave heading:

- Waves from different directions excite different vessel motion responses:
 - Surge (fore-aft) responds primarily to head seas (0°) and following seas (180°)
 - Sway (lateral) responds primarily to beam seas (90° and 270°)
 - Roll is excited by beam seas, Pitch by head/following seas
- Phase relationships between motions encode arrival direction — the time delay between surge and sway oscillations changes continuously with wave heading
- Energy ratios (`vel_diff_sway_surge_time_energy_ratio`) capture whether wave energy is primarily longitudinal or lateral — directly discriminates between quadrants
- Cross-spectral phase features (`xspec_phase_lag`) measure the phase relationship at the dominant wave frequency — this varies smoothly and monotonically with heading within each quadrant
- System linking features (`link_phase_lag_*_vs_sea`) directly encode the lag between vessel motion and sea surface elevation at the dominant frequency — this is a direct function of wave direction

The 156 features were designed to capture these physical relationships. The RFE process then selects the minimal subset that preserves the direction information while removing noise.

Physics: Features for Fine Angular Regression

Within a 90° quadrant, features must discriminate fine angular differences:

Feature	Physical Mechanism	How It Discriminates Within a Quadrant
pos_ellipse_orientation_deg_cos	Orientation of the surge/sway motion ellipse	The ellipse rotates continuously with wave heading — provides direct angular information
sea_elevation_peak_period_Tp_s	Dominant wave period from sea spectrum	Combined with motion features, disambiguates wave conditions at different angles
link_resonance_proximity_*_vs_sea_s	Distance between vessel natural period and wave period	Affects response amplitude/phase, varies continuously with heading
pos_xspec_phase_lag_roll_pitch_deg_sin/cos	Roll-pitch cross-spectral phase	Phase relationship changes smoothly with heading — sin/cos encoding preserves continuity
vel_diff_hilbert_phase_lag_mean_deg_sin	Sway-surge velocity phase lag	Instantaneous phase difference tracks heading variation within the quadrant

- The 21 regression features are the same for all 4 quadrants — the models learn quadrant-specific mappings from the same feature space, because the physical relationships between features and heading angle differ across quadrants
- All 21 features use 'none' for transform and scaling — XGBoost and Decision Trees are scale-invariant and work directly with raw feature values

21 Regression Features (All Quadrants Same)

#	Feature Name	Physical Meaning
1	vel_diff_sway_surge_time_energy_ratio	Sway vs surge kinetic energy fraction
2	vel_diff_hilbert_phase_lag_mean_deg_sin	Sin of sway-surge velocity phase lag
3	vel_diff_xspec_phase_lag_waveband_deg_cos	Cos of cross-spectral phase in wave band
4	pos_sway_std_m	Lateral position std (m)
5	acc_surge_std_ms2	Surge acceleration std (m/s ²)
6	acc_surge_variance_m2s4	Surge acceleration variance (m ² /s ⁴)
7	pos_xcorr_roll_pitch_coeff	Roll-pitch correlation coefficient
8	pos_pitch_spectral_total_energy	Total pitch spectral energy
9	pos_xspec_phase_lag_roll_pitch_deg_sin	Sin of roll-pitch cross-spectral phase
10	pos_xspec_phase_lag_roll_pitch_deg_cos	Cos of roll-pitch cross-spectral phase
11	sea_elevation_significant_height_Hs_m	Significant wave height (m)

All 21 features use transform='none', scaling='none'. Tree-based models work on raw values. Note: regression uses sea_elevation_significant_height_Hs_m (not in circular set); circular uses pos_xspec_coherence_roll_pitch_max (not in regression set).

21 Regression Features (All Quadrants Same)

#	Feature Name	Physical Meaning
12	pos_dominant_motion_dir_deg_cos	Cos of dominant motion direction
13	pos_roll_pitch_phase_change_low_high_freq_rad	Roll-pitch phase change across wave band
14	pos_sway_surge_xcorr_max_lag_samples	Peak cross-correlation lag (sway-surge)
15	pos_surge_hilbert_envelope_skewness	Surge envelope skewness
16	pos_ellipse_orientation_deg_cos	Cos of motion ellipse orientation
17	sea_elevation_peak_period_Tp_s	Dominant wave period (s)
18	link_resonance_proximity_roll_vs_sea_s	Roll vs sea period proximity (s)
19	link_phase_lag_at_peak_freq_roll_vs_sea_deg_cos	Cos of roll-sea phase lag
20	link_resonance_proximity_surge_vs_sea_s	Surge vs sea period proximity (s)
21	link_phase_lag_at_peak_freq_surge_vs_sea_deg_sin	Sin of surge-sea phase lag

All 21 features use transform='none', scaling='none'. Tree-based models work on raw values. Note: regression uses sea_elevation_significant_height_Hs_m (not in circular set); circular uses pos_xspec_coherence_roll_pitch_max (not in regression set).

Production Configuration (Hierarchical)

Commands:

```
Train:  python main.py --mode train --input "path/to/simulation_data/"
Predict: python main.py --mode predict --input "path/to/sample.csv"
```

Production Artifacts (all from same training run):

File	Description
models/classifier.pkl	ExtraTrees 500 trees, 36 features → 4 quadrants
models/classifier_preprocessor.pkl	Fitted normalization for 36 classification features
models/Q1_regressor.pkl	XGBoost, 300 trees, 21 features → sin/cos
models/Q2_regressor.pkl	XGBoost, 300 trees, 21 features → sin/cos
models/Q3_regressor.pkl	XGBoost, 300 trees, 21 features → sin/cos
models/Q4_regressor.pkl	Decision Tree, 21 features → sin/cos
models/regression_preprocessor.pkl	Fitted normalization for 21 regression features

Source Code:

- src/features.py (1,593 lines) — Same 156-feature extraction as circular pipeline
- src/trainer.py — Two-stage training: classification RFE + per-quadrant regression
- src/predictor.py — Cascade inference: classify → route → predict → reconstruct
- configs/models_config.json — All 5 model configurations in one file
- configs/hierarchy_features.json — 36 classification + 21 regression features

Ref: *hierarchical_quadrant_cascaded_pipeline/configs/*

Retrain-to-Predict: Both Wave Heading Approaches

Circular Regression (2 artifacts):

- Train: `python main.py --mode train --input "csvs/"` → saves `ensemble_model.pkl` + `preprocessor.pkl`
- Predict: `python main.py --mode predict --input "new.csv"` → extract 22 features → normalize → predict `[sin,cos]` → `atan2` → heading°

Hierarchical Cascade (7 artifacts):

- Train: `python main.py --mode train --input "csvs/"` → saves classifier + 4 regressors + 2 preprocessors
- Predict: `python main.py --mode predict --input "new.csv"` → extract features → classify quadrant → route → predict `[sin,cos]` → `atan2` → heading°

Both approaches:

- Read CSV time-series → extract features (same `features.py`) → apply saved preprocessor → predict → reconstruct angle
- All `pkl` files must come from the same training run — replacing one without the others will produce incorrect predictions
- Feature extraction uses the same 1,593-line `features.py` in both pipelines
- Input CSVs must contain: Time, Surge, Sway, Roll, Pitch, Surge Acceleration, Sway Acceleration, Sea Elevation
- Units: linear columns in feet (converted to metres internally), angular columns in degrees, time in seconds

Summary

Without RFE (naive direct train/test):

- Train model on all 172 features, evaluate on test set, select features that look best
- Information leakage: features are selected because they correlate with the specific test partition
- Bias: the model overfits to which features happen to work on this particular data split
- Result on truly unseen data will be worse than the test performance suggests

With RFE (3-partition design):

- 3 partitions: Train (fit), Validation (decide), Test (report only)
- Test set is NEVER used in any decision — only for final, unbiased evaluation
- Features must prove value on the independent validation set
- Selected features capture genuine physical signal, not noise patterns

Pipeline	Validation Metric	Test Metric	Gap
Circular Regression CMAE	0.438°	0.623°	+0.185° (small)
Classification Accuracy	99.83%	100.0%	Test better (no leakage)
Circular Max Error	10.54°	21.31°	+10.77° (tail sensitivity)

- Small validation-to-test gaps confirm the model generalizes well to unseen data
- Classification test accuracy (100.0%) exceeds validation (99.83%) — no test leakage
- Max error gap (10.54° → 21.31°) is larger because max error is sensitive to a single outlier — this is expected behavior, not a sign of overfitting

All Pipelines Summary

Pipeline	Input	Features	Model	Key Metric	Experiments
Fatigue Damage	5 env. params	6	MLP (46 neurons)	$R^2 = 0.8086$	3,525
Significant Wave Height	6 motion STDs	6	Random Forest (100 trees)	$R^2 = 0.9992$, RMSE = 0.44 ft	245
Heading (Circular)	22 from 156	22	ExtraTrees (600 trees)	CMAE = 0.62°, Max = 21.31°	~1.08M fits
Heading (Hierarchical)	36 + 21	36 + 21	ExtraTrees + XGBoost/DT	Max = 12.50° (Q4)	36,752

Fatigue R^2

0.8086

Hs RMSE

0.44 ft

Heading CMAE

0.62°

Heading Max

12.50°

- Total experiments across all pipelines: 3,525 (fatigue) + 245 (Hs) + ~1.08M fits (circular RFE) + 36,752 (hierarchical regression)
- All pipelines include: production source code, trained model artifacts, configuration files, and complete technical documentation
- All data in Imperial units (ft, deg, s) at input, with internal SI conversion where needed
- Each pipeline has a retrain-to-predict workflow: place data → run train command → artifacts saved → run predict command

Technical Data & Artifact Index

[fatigue_prediction_pipeline/technical_data/](#)

- 00_source_code/ — 13 converted notebook scripts
- 01_data_characterization/ — Training and test datasets
- 02_feature_engineering/ — Engineered feature snapshots
- 03_ml_benchmarking/ — 198 ML experiment results
- 04_dl_architecture_search/ — 3,327 DL experiment results
- 05_selected_model_artifacts/ — Production model metrics and comparisons

[significant_wave_height_prediction_pipeline/technical_data/](#)

- 00_source_code/ — Original notebooks converted to .py
- 01_convergence_analysis/ — Data validation results
- 02_feature_analysis/ — Correlation reports
- 03_model_selection_experiments/ — 245 experiment results

[wave_heading_prediction_pipeline/ technical_data/](#)

- circular_regression_full_heading/
- 03_recursive_feature_elimination_tracking/ — 166-round RFE trajectory CSV
- 04_selected_model_round_146_artifacts/ — 22-feature model metrics
- hierarchical_quadrant_cascaded_pipeline/
- 04_classification_rfe_171_rounds/ — 171-round classification RFE CSV
- 05_best_round_137_selection/ — 36-feature classifier metrics
- 08_regression_comparative_analysis/ — 36,752 regression experiments across 4 datasets

AI-Driven Sample Space Analysis for Offshore Riser Behavior Modeling

3 prediction pipelines | Fatigue Damage | Significant Wave Height | Wave Heading

3,525 + 245 + ~1.08M + 36,752 experiments

Complete source code, trained artifacts, configuration, and technical documentation